



OpenEden / Hybond

May 21, 2026





Table of Contents

Summary

Overview

Issues

[WP-H1] Address the root cause of sharesPerToken invariance

[WP-H2] De-KYC'd redeem sender deadlocks both settlement and emergency cancel — incomplete fix to prior WP-M, plus new Token-side KYC gate makes cancel revert too

[WP-H3] Flawed management fee tracking logic corrupts sharesPerToken, leading to incorrect deposit and redeem calculations.

[WP-M4] Sudden drop in sharesPerToken after processDepositQueue().

[WP-M5] cancelRedeem() and revertRedeemToPending() trigger a precipitous drop in sharesPerToken.

[WP-M6] processDepositQueue uses live token.totalSupply(), producing intra-batch price drift that advantages earlier vs later depositors in the same batch

[WP-M7] setSnapshotRatio caps input at 1e18, blocking MAINTAINER from replicating legitimate protocol states where _sharesPerToken() > 1e18

[WP-M8] PriceOracle.confirmPrice sets round.updatedAt = observedAt, forcing maxStalePeriod to be set wider than any real freshness threshold — defeating the guard it's meant to enforce

[WP-M9] Redeem fee is priced at pending → final migration time, not at user-request time — MAINTAINER-initiated updateRedeemFeeRate retroactively changes the fee queued users will pay

[WP-M10] Express.updateEpoch() fails to account for the price discrepancy between off-chain BNY shares and Hybond token.

[WP-M11] When the pendingRedeemQueue is non-empty, the offchainShares used to accrue the DailyMgtFee may be artificially inflated, as it includes off-chain BNY shares that have already been marked for sale.

[WP-M12] Management Fee Accrual Depends on Call Frequency Instead of Elapsed Time

[WP-M13] Deposit Queue Head-Blocking DOS

[WP-M14] PriceOracle Propose/Confirm Not Bound to Specific Value



[WP-M15] `cancelDeposit` is intended to refund the fee, but the fee has already been transferred to `txFeeTo` during `requestDeposit`.

[WP-L16] `updateMgtFeeTo` lacks on-chain enforcement of its documented invariants, silently corrupting `totalMgtFeeUnclaimed` when rotated with in-flight redemptions

[WP-L17] `maxStalePeriod` is not initialized and defaults to 0, silently disabling price staleness checks

[WP-L18] `processPendingRedeems()` should likely revert when `sender == mgtFeeTo` && `shareAmount > totalMgtFeeUnclaimed`.

[WP-L19] If `Express.circulatingSupply()` is no longer being used, consider removing the `circulatingSupply()` function.

[WP-L20] The `depositInfo[_account]` returned by `Express.getDepositUserInfo()` is an aggregate of multiple `_asset` amounts, each potentially having different decimals and valuations.

[WP-L21] `PriceOracle` Deviation Formula Uses Symmetric Average Instead of Base-Relative

[WP-L22] `PriceOracle.confirmPrice()` fails to specify the expected `observedAt` and `proposedAt`.

[WP-L23] The `updateRedeemAsset` and `initialize` functions fail to verify whether `_address` is supported by `assetRegistry`.

[WP-L24] `DoubleQueueModified` derives from an older version of the `OpenZeppelin DoubleEndedQueue` and inherits some of its original issues.

[WP-L25] `PriceOracle` uses unique storage slots for each round to preserve historical data, yet it lacks the `AggregatorV3Interface.getRoundData(uint80 _roundId)` interface.

[WP-I26] Operational hazard: `updateOffchainShares` accepts arbitrary values while queues are non-empty — recommend on-chain queue-empty guard

[WP-I27] Operational hazard: `Token.setKycManager` has no queue-empty / cross-contract consistency guard — recommend mirroring `Express`'s rotation discipline on-chain

[WP-I28] Both `depositMaxDeviationBps` and `redeemMaxDeviationBps` default to zero on `initialize`, and the setters require queues empty — deployment-day self-DoS trap

[WP-I29] Cancelled first deposit leaves `firstDeposit` permanently set

[WP-I30] `AssetRegistry` Initializer Grants `MAINTAINER_ROLE` to `msg.sender`

[WP-I31] Operator Can Inflate Management Fees via `updateEpoch` Override

[WP-I32] A single price for the buy and sell price of the BNY ETF may not be enough.



[WP-I33] The Chainlink-compatible answer type is int256, which is inconsistent with the current PriceOracle implementation.

[WP-N34] Initializable and UUPSUpgradeable in @openzeppelin/contracts-upgradeable have been deprecated since version 5.5.0.

[WP-G35] Values that do not change, like PriceOracle.decimalsValue, should be declared as immutable to avoid unnecessary SLOAD gas costs.

Appendix

Disclaimer



Summary

This report has been prepared for OpenEden / Hybond smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



Overview

Project Summary

Project Name	OpenEden / Hybond
Codebase	https://github.com/OpenEdenHQ/openeden.hybond.audit
Commit	338039ff304ba18a080ce7850f5c4274e874ef8a
Language	Solidity

Audit Summary

Delivery Date	May 21, 2026
Audit Methodology	Static Analysis, Manual Review
Total Issues	35



[WP-H1] Address the root casue of sharesPerToken invariance

High

Issue Description

We want the `offchainShares` to be maintained mostly onchain.

There should be no change of `sharesPerToken` during deposit and redeem; `OffchainShares` should not require manual update unless extremely rare events happen (e.g., share split).

There can be a method to change the `offchainShares` amount, but it should almost never be called.

Properly tracking `sharesPerToken` should involve dividing `totalUnderlyingAsset`—the total amount of off-chain BNY shares—by the total supply of the `token`.

As the management token mints, `sharesPerToken` will automatically dilute.

Deposit and mint operations should increment `totalUnderlyingAsset` (BNY EFT Shares) by the actual amount of off-chain BNY shares deposited, while minting an amount of `token` calculated based on the current `sharesPerToken` and the actual amount of off-chain BNY shares (converted from the user-deposited asset off-chain BNY shares), which should be a parameter of `processDepositQueue`.

`processDepositQueue()`:

- Added `startId` and `batchTotalOffchainShares` parameters.
- All deposit processes within a single `processDepositQueue()` call will utilize the same BNY Shares/USDC exchange rate.

`requestRedeem(_to, _shareAmount)` should record the corresponding off-chain BNY shares (calculated at the `requestRedeem()` time and saved to storage):

- Produce effect: divisor of `_sharesPerToken()` is reduced by `_shareAmount`
- Record `offChainShares` corresponding to `pendingId` in storage
- `offchainShares -= offChainShares`



- Delete `Express.snapshotRatios`

`processPendingRedeems()`:

- Added `startId` and `totalUSDC` parameters.
- We no longer track the BNY Shares to USDC exchange rate in `processPendingRedeems()`. `processRedeemQueue(startId, len, totalUSDC)` needs to have a new parameter for the actual amount of USDC redeemed from the BNY ETF shares.
- `processRedeemQueue` must specify the exact range (startId, len)
- the total USDC amount will be distributed among the batch according to their recorded offchain shares of each redeem
- the same batch should correspond to the actual batch being processed off-chain (should be the same day)

We also want you to remove `revertRedeemToPending()` so that the redeem queue is always ordered by request time.

`sharesPerToken` remains constant during deposit and redemption operations.

Removal of BNY Shares/USDC Rate Oracle

This new design doesn't use the on-chain BNY Shares/USDC rate oracle anymore; all token mint/burn use the **ACTUAL** off-chain BNY shares amount and the **ACTUAL** USDC amount from redemption (sourced from the operator).

Status

✓ Fixed



[WP-H2] De-KYC'd redeem sender deadlocks both settlement and emergency cancel — incomplete fix to prior WP-M, plus new Token-side KYC gate makes cancel revert too

High

Issue Description

Prior context. Prior reported the FIFO-blocking shape — banned or de-KYC'd user at the head of either queue blocks every entry behind them. The recommendation was a `try/catch`-style escrow on processing failure. The team's actual fix was partial: `_refundOrEscrow` was added for the **cancel** path and routes to `redeemEscrowBalance` only when `token.isBanned(_sender)`. The **de-KYC** branch was not added, so de-KYC'd users still hit `token.safeTransfer(_sender, _amount)` directly.

Round 2 then added a NEW Token-side KYC gate in `Token._update`. Before round 2, `safeTransfer` from Express to a de-KYC'd user would still succeed (Token didn't check KYC). After round 2, that transfer reverts inside Token. The previously-recoverable Medium became unrecoverable on-chain — which is why the severity is now HIGH.

The round-2 KycManager extraction added a KYC gate to `Token._update` (when Token is in permissioned mode). This gate now sits on every HYBOND mint, burn, and transfer — including the cancel-path refund that was supposed to be the recovery for exactly this situation.

The failure mode chains across three layers:

1. **Settlement blocked.** A user submits `requestRedeem` while KYC'd. Their entry is in `pendingRedeemQueue` (or `redeemQueue`). KYC gets revoked. `processPendingRedeems` calls `_validateKyc(sender, receiver)` (Express's check) and reverts. Same for `processRedeemQueue` and `cancelPendingRedeem` paths that re-validate KYC.
2. **Cancel blocked.** MAINTAINER tries `cancelPendingRedeem(1)` to unblock the queue head. The cancel calls `_refundOrEscrow(sender, tokenAmount)`, which falls through to `IERC20(token).safeTransfer(sender, ...)` because the user is not banned. That transfer enters `Token._update`, which now reads `kycManager` and reverts `NotKycd(sender)`.
3. **Escrow misses the case.** `_refundOrEscrow` only routes to `redeemEscrowBalance` when `token.isBanned(sender)`. De-KYC'd is not banned. So the de-KYC'd user never gets their



tokens back into escrow either; they sit at the head of the queue holding everyone else hostage.

Real-world compliance events (sanctions, account closure, jurisdiction changes) routinely cause KYC revocation. Each such event freezes every redemption behind the affected user until WHITELIST_ROLE re-grants KYC. There is no on-chain workaround.

PoC

A repeatable Hardhat test was added at `test/unit/Express.dekycDeadlock.test.ts` (5 tests, all passing on `wp/5 @ 2086a82`):

```
Express - De-KYC redeem deadlock (PoC for WP-H)
Pending queue path
  settlement reverts when redeemer is de-KYC after enqueue
  cancelPendingRedeem reverts because refund safeTransfer hits Token KYC gate
  queue is wedged: a second user behind the de-KYC head also cannot settle
Final queue path (after T+2)
  cancelRedeem reverts on de-KYC even after entry has moved to final queue
Recovery via re-KYC works
  re-granting KYC unblocks both settlement and cancel paths
5 passing
```

The first three tests cover the pending-queue side; the fourth covers the final-queue side after T+2 settlement. The fifth confirms the only on-chain unblock is re-granting KYC. The "queue is wedged" test specifically demonstrates the FIFO blocking — user2 cannot make progress because user1 sits at the head with revoked KYC.

Reproduce with: `npx hardhat test test/unit/Express.dekycDeadlock.test.ts`.

Vulnerable Code

`_refundOrEscrow` only escrows on the banned condition, missing de-KYC'd users:



```
1612 function _refundOrEscrow(address _sender, uint256 _amount) internal {
1613     if (token.isBanned(_sender)) {
1614         redeemEscrowBalance[_sender] += _amount;
1615         emit RedeemEscrowIn(_sender, _amount);
1616     } else {
1617         IERC20(address(token)).safeTransfer(_sender, _amount);
1618     }
1619 }
1620
```

Token._update now rejects transfers to/from non-KYC'd addresses, so the **safeTransfer** branch reverts:

```
257 }
258
259 // KYC gate (permissionless when kycManager == address(0))
260 address mgr = kycManager;
261 if (mgr != address(0)) {
262     if (from != address(0) && !IKycManager(mgr).isKycd(from)) revert
    NotKycd(from);
263     if (to != address(0) && !IKycManager(mgr).isKycd(to)) revert NotKycd(to);
264 }
265
```

The revert happens inside **cancelPendingRedeem**'s refund leg:

```
1095 // Restore mgtFeeUnclaimed if this was a fee redeem
1096 if (sender == mgtFeeTo) {
1097     totalMgtFeeUnclaimed += tokenAmount;
1098 }
1099
1100 unchecked {
1101     --_len;
1102 }
1103
1104 _refundOrEscrow(sender, tokenAmount);
1105
1106 emit CancelPendingRedeem(sender, receiver, tokenAmount, id);
```

The same vector hits **cancelRedeem** at line 1209.

Recommendation



Route to escrow whenever a direct token transfer back to the sender would fail. The simplest fix extends the escrow trigger to cover de-KYC:

```
function _refundOrEscrow(address _sender, uint256 _amount) internal {
    bool blocked = token.isBanned(_sender);
    if (!blocked && kycManager != address(0)) {
        blocked = !IKycManager(kycManager).isKycd(_sender);
    }
    if (blocked) {
        redeemEscrowBalance[_sender] += _amount;
        emit RedeemEscrowIn(_sender, _amount);
    } else {
        IERC20(address(token)).safeTransfer(_sender, _amount);
    }
}
```

claimRedeemEscrow should then either be reachable by an OPERATOR on the user's behalf (already supported via the **_account** parameter) or include a parallel KYC-gated path. Verify the equivalent edge case on the deposit-cancel side (**cancelDeposit** → **depositEscrowBalance** doesn't go through Token, so it's unaffected — but worth a sanity check on **claimRedeemEscrow** user-side flow).

Status

✓ Fixed



[WP-H3] Flawed management fee tracking logic corrupts `sharesPerToken`, leading to incorrect deposit and redeem calculations.

High

Issue Description

When `mgtFeeTo` transfers Hybond tokens to another address or the `mgtFeeTo` address is updated, the ratio between the Hybond token balance and off-chain BNY shares—defined by `_sharesPerToken()`—becomes desynchronized.

`Express` implements the `DailyMgtFee` by minting `dailyFee` amounts of Hybond tokens to `mgtFeeTo`, effectively diluting the token supply. Consequently, the Hybond `totalSupply()` exceeds the total off-chain BNY shares.

However, the current implementation naively calculates the inflated token amount based solely on `token.balanceOf(mgtFeeTo)`. If `mgtFeeTo` transfers tokens or the `Express.mgtFeeTo` address is changed, this calculation underestimates the actual inflation. This results in users receiving an excessive amount of off-chain BNY shares during redemption.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/doc/plan/management-fee.md#L1-L30>



```
1  # Overview
2
3  Hybond is a token issued via `Token.sol`, used to tokenize real-world assets: a BNY
   ETF Fund.
4
5  ## Token Mint Workflow (Express.sol:requestDeposit)
6
7  @@ 7,12 @@
13
14  ## Token Redeem Workflow (Express.sol:requestRedeem)
15
16  @@ 16,23 @@
24
25  ## Management Fee
26
27  1. Fund operator calls `updateEpoch` (OPERATOR_ROLE) daily to calculate and mint that
   day's management fee to `mgtFeeTo` based on `circulatingSupply`.
28  2. `updateEpoch` / `updateEpochAdjust` revert when `mgtFeeRate == 0`.
29  3. Off-chain BNY shares correspond to `circulatingSupply`, not `totalSupply`.
30  4. `totalSupply = circulatingSupply + totalRedeemQueueShares + mgtFeeTo balance`.
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/Express.sol#L1461-L1469>

```
1461  /**
1462   * @notice Calculate the current shares-per-token ratio in 1e18 precision
1463   * @dev Uses circulating supply to exclude claimed management-fee inventory and
   final redeem queue inventory
1464   */
1465   function _sharesPerToken() internal view returns (uint256 ratio) {
1466       uint256 totalSupply = IERC20(address(token)).totalSupply();
1467       if (totalSupply == 0) return 1e18;
1468       ratio = Math.mulDiv(circulatingSupply(), 1e18, totalSupply);
1469   }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/Express.sol#L1367-L1378>

```

1367  /**
1368  * @notice Get circulating token supply
1369  * @dev Circulating = Total Supply - Tokens in redeem queue - Tokens at mgtFeeTo
1370  * @return supply Circulating token supply
1371  */
1372  function circulatingSupply() public view returns (uint256 supply) {
1373      IERC20 tokenERC20 = IERC20(address(token));
1374      uint256 totalSupply = tokenERC20.totalSupply();
1375      uint256 tokensAtMgtFeeTo = mgtFeeTo != address(0) ? tokenERC20.balanceOf-
(mgtFeeTo) : 0;
1376
1377      supply = totalSupply - totalRedeemQueueShares - tokensAtMgtFeeTo;
1378  }

```

```

406  /**
407  * @notice Update the management fee recipient address
408  * @param _address The new management fee recipient address
409  */
410  function updateMgtFeeTo(address _address) external onlyRole(MAINTAINER_ROLE) {
411      if (_address == address(0)) revert InvalidAddress();
412      mgtFeeTo = _address;
413      emit UpdateMgtFeeTo(_address);
414  }

```

```

1325  /**
1326  * @notice Shared epoch update logic for default and adjusted paths
1327  * @dev Mints the daily management fee immediately to mgtFeeTo.
1328  * @param _circulatingSupply Manual circulating supply value
1329  * @param _useOverride Whether to use manual circulating supply
1330  */
1331  function _updateEpochInternal(uint256 _circulatingSupply, bool _useOverride)
internal {
1332      if (mgtFeeRate == 0) revert MgtFeeDisabled();
1333      if (lastUpdateTS != 0 && block.timestamp < lastUpdateTS + timeBuffer) {
1334          revert UpdateTooEarly(block.timestamp);
1335      }
1336
1337      epoch++;
1338
1339      uint256 circulating;
1340      if (_useOverride) {
1341          uint256 totalSupply = IERC20(address(token)).totalSupply();
1342          if (_circulatingSupply > totalSupply) revert InvalidInput(_circulatingSupply);

```



```
1343     circulating = _circulatingSupply;
1344 } else {
1345     circulating = circulatingSupply();
1346 }
1347
1348 uint256 dailyFee = _trim(_calculateDailyMgtFee(circulating));
1349 if (dailyFee > 0) {
1350     if (mgtFeeTo == address(0)) revert InvalidAddress();
1351     token.mint(mgtFeeTo, dailyFee);
1352 }
1353
1354 lastUpdateTS = block.timestamp;
1355 emit UpdateEpoch(unclaimedMgtFee, dailyFee, epoch, circulating);
1356 }
```

Recommendation

To accurately track the management fee balance, do not rely on the balance of a specific address. Instead, utilize a dedicated storage variable.

However, this does not address the root cause of the issue.

Properly tracking **sharesPerToken** should involve dividing **totalUnderlyingAsset**—the total amount of off-chain BNY shares—by the total supply of the **token**.

As the management token mints, **sharesPerToken** will automatically dilute.

Deposit and mint operations should increment **totalUnderlyingAsset** (BNY EFT Shares) by the actual amount of off-chain BNY shares deposited, while minting an amount of **token** calculated based on the current **sharesPerToken** and the actual amount of off-chain BNY shares (converted from the user-deposited asset off-chain BNY shares), which should be a param of **processDepositQueue**.

requestRedeem should record the corresponding off-chain BNY shares (calculated at the **requestRedeem** time and saved to storage), and when processed, transfer the actual amount of redeemed USDC back to the user.

processRedeemQueue need to have a new param for the actual amount of USDC redeemed from the BNY ETF shares.

sharesPerToken remains constant during deposit and redemption operations.



With the updated **sharesPerToken** design, calculate the management fee as follows:

Assume a management fee of 0.01% per epoch.

The amount of tokens minted for the management fee is $(\text{totalSupply} - \text{pendingRedeem}) * 0.01\%$.

pendingRedeem (the unit is token, not underlying asset unit) is excluded from the fee base, as pending redemptions should not incur management fees.

Status

✓ Fixed



[WP-M4] Sudden drop in sharesPerToken after processDepositQueue().

Medium

Issue Description

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/docs/management-fee.md#L1-L35>

```
1  @@ 1,25 @@
26
27  ## Token Mint Workflow (`Express.sol::requestDeposit`)
28
29  1. User calls `requestDeposit` to deposit USDC to the contract at T+0.
30  2. Fund operator transfers the USDC to buy BNY ETF Fund off-chain at T+0 EOD.
31  3. At T+2, BNY emails us the share price and share amount of the BNY ETF Fund.
32  4. Fund operator updates the share price in `PriceOracle.sol` via `proposePrice` and `confirmPrice`.
33  5. Fund operator calls `processDepositQueue` (MAINTAINER_ROLE) to mint Hybond tokens to users.
34  6. Fund operator calls `proposeOffchainShares` (OPERATOR_ROLE) and CONFIRM_ROLE calls `confirmOffchainShares` to sync the new BNY share balance.
35  7. User receives Hybond tokens in their wallet.
```

`processDepositQueue()` only increases `token.totalSupply()` and does not affect `offchainShares` (i.e., `offchainShares` remains constant after `processDepositQueue()`).

Between steps 5 and 6 of the "Token Mint Workflow (Express.sol::requestDeposit)" described in the previous documentation, `sharesPerToken` exists in an artificially low state.

Example:

Given:

- `token.totalSupply()`: 10,000.0
- `totalRedeemQueueShares`: 0
- `offchainShares`: 10,000.0
- `convertFromUnderlying()` price: 1.0



- `getPrice()`: 1.0
- `depositQueue` has 3 `DepositData` entries, each with a `netAssets` of 2,000.0

When: `processDepositQueue(3)`

- 1st iteration of `processDepositQueue()` (processing the 1st `DepositData`):
 - Here, `_sharesPerToken()` is
$$\frac{\text{offchainShares} \times 10^{18}}{\text{token.totalSupply()} - \text{totalRedeemQueueShares}} = \frac{10000 \times 10^{18} \times 10^{18}}{10000 \times 10^{18} - 0} = 10^{18}$$
 - `mintedAmount = _calculateMintAmount(asset, netAssets) = _calculateMintAmount(asset, 2,000.0)` yields `2_000e18`
 - `token.mint(receiver, mintedAmount)` mints `2_000e18`, increasing `token.totalSupply()` from `10_000e18` to `12_000e18`
- 2nd iteration of `processDepositQueue()` (processing the 2nd `DepositData`):
 - Here, `_sharesPerToken()` is
$$\frac{\text{offchainShares} \times 10^{18}}{\text{token.totalSupply()} - \text{totalRedeemQueueShares}} = \frac{10000 \times 10^{18} \times 10^{18}}{12000 \times 10^{18} - 0} = 0.8333333333333333 \times 10^{18} \text{ (result of } \frac{10 \times 10^{18}}{12} \text{ rounded down)}$$
 - `==16.666666666666667% less than 1e18==`
 - `mintedAmount = _calculateMintAmount(asset, netAssets) = _calculateMintAmount(asset, 2,000.0)` yields
$$\frac{2000 \times 10^{18} \times 10^{18}}{8333333333333333} = 2400.000000000000000960 \times 10^{18}$$
 - `==Approx. 400e18 more than the 1st iteration==`
 - `token.mint(receiver, mintedAmount)` mints `2_400.000000000000000960e18`, increasing `token.totalSupply()` from `12_000e18` to `14_400.000000000000000960e18`
- 3rd iteration of `processDepositQueue()` (processing the 3rd `DepositData`):



- Here, `_sharesPerToken()` is

$$\frac{\text{offchainShares} \times 10^{18}}{\text{token.totalSupply()} - \text{totalRedeemQueueShares}} = \frac{10000 \times 10^{18} \times 10^{18}}{14_400.0000000000000000960 \times 10^{18} - 0} = 0.6944444444444444 \times 10^{18}$$

- `==30.5555555555555556% less than 1e18==`

- `mintedAmount = _calculateMintAmount(asset, netAssets)`

`= _calculateMintAmount(asset, 2,000.0)` yields

$$\frac{2000 \times 10^{18} \times 10^{18}}{6944444444444444} = 2880.0000000000000001843 \times 10^{18}$$

- `==Approx. 880e18 more than the 1st iteration==`

- `token.mint(receiver, mintedAmount)` mints `2_880.0000000000000001843e18-`, increasing `token.totalSupply()` from `14_400.000000000000000960e18` to `17_280.0000000000000002803e18`

Deposits of the same amount in the same batch result in different amounts of Hybond `token`. In the example above, except for the first iteration of `processDepositQueue()`, the calculations used an artificially low `_sharesPerToken()`.

Note: The example above is only one scenario where `sharesPerToken` is used; other functionalities (e.g., other contracts or off-chain systems reading `sharesPerToken()`) may also be affected by this artificially low value.

Current design relies on `proposeOffchainShares` and `confirmOffchainShares` to be executed right after `processDepositQueue` for the integrity of the `_sharesPerToken`.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L662-L693>

```
662  /**
663   * @notice Process queued deposit requests
664   * @param _len Number of requests to process (0 = process all)
665   */
666   function processDepositQueue(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
667     _len = _validateQueueProcessing(depositQueue.length(), _len);
668
669     for (uint256 count = 0; count < _len; ) {
```



```
670 bytes memory data = depositQueue.front();
671 (
672     address asset,
673     address sender,
674     address receiver,
675     uint256 netAssets,
676     uint256 feeAmt,
677     bytes32 prevId
678 ) = _decodeDepositData(data);
679
680 _validateKyc(sender, receiver);
681
682 depositQueue.popFront();
683 depositInfo[receiver][asset] -= netAssets;
684 unchecked {
685     ++count;
686 }
687
688 uint256 mintedAmount = _calculateMintAmount(asset, netAssets);
689 token.mint(receiver, mintedAmount);
690
691 emit ProcessDeposit(asset, sender, receiver, netAssets, mintedAmount,
692     feeAmt, prevId);
693 }
```

```
608 /**
609  * @notice Calculate token mint amount from net asset amount
610  * @param _asset Asset token address
611  * @param _netAssets Net asset amount (after fees)
612  * @return mintAmount Token amount to mint
613  */
614 function _calculateMintAmount(address _asset, uint256 _netAssets) internal view
615     returns (uint256 mintAmount) {
616     uint256 amount = convertFromUnderlying(_asset, _netAssets);
617     uint256 price = getPrice();
618     uint256 tokenPrice = Math.mulDiv(price, _sharesPerToken(), 1e18);
619     mintAmount = _trim(Math.mulDiv(amount, 1e18, tokenPrice));
620 }
```



```
1585  /**
1586  * @notice Calculate the current shares-per-token ratio in 1e18 precision
1587  * @dev Formula: offchainShares / (totalSupply - totalRedeemQueueShares), scaled
1588  *      by 1e18.
1589  *      Denominator excludes the final redeem queue (those shares are already
1590  *      priced).
1591  *      Pending-redeem-queue shares are included in the denominator.
1592  *
1593  *      Returns the 1e18 fallback when either denom is zero (bootstrap / full
1594  *      exit) or
1595  *      offchainShares has not yet been set by the bot (pre-first-sync). In the
1596  *      pre-sync
1597  *      window deposits and redeems settle at 1:1 against the underlying asset,
1598  *      which is
1599  *      economically fair: depositors mint at 1:1 and redeemers get their deposit
1600  *      back.
1601  *      updateEpoch is a no-op in this window because dailyFee = offchainShares *
1602  *      rate = 0.
1603  */
1604  function _sharesPerToken() internal view returns (uint256 ratio) {
1605      uint256 totalSupply = IERC20(address(token)).totalSupply();
1606      uint256 denom = totalSupply - totalRedeemQueueShares;
1607      if (offchainShares == 0 || denom == 0) return 1e18;
1608      ratio = Math.mulDiv(offchainShares, 1e18, denom);
1609  }
```

```
1481  /*//////////////////////////////////////
1482  OFFCHAIN SHARES MANAGEMENT
1483  //////////////////////////////////////////*/
1484
1485  /**
1486  * @notice Propose a new offchain BNY share value
1487  * @dev Latest-wins: overwrites any prior unconfirmed proposal
1488  * @param _supply Proposed value (18-decimal convention, must match HYBOND
1489  *      totalSupply scale)
1490  */
1491  function proposeOffchainShares(uint256 _supply) external onlyRole(OPERATOR_ROLE)
1492  {
1493      if (_supply == 0) revert InvalidAmount();
1494      proposedOffchainShares = _supply;
1495      emit ProposeOffchainShares(_msgSender(), _supply);
1496  }
1497
1498  /**
1499  * @notice Confirm the pending proposed offchain share value
```



```
1498 * @dev Caller must read proposedOffchainShares off-chain and echo it here; any
      mismatch
1499 *         reverts, ensuring the confirmer independently verifies the value proposed
      by OPERATOR_ROLE.
1500 * @param _supply Must equal proposedOffchainShares; anything else reverts
1501 */
1502 function confirmOffchainShares(uint256 _supply) external onlyRole(CONFIRM_ROLE) {
1503     uint256 pending = proposedOffchainShares;
1504     if (pending == 0) revert InvalidAmount();
1505     if (_supply != pending) revert InvalidInput(_supply);
1506
1507     uint256 previous = offchainShares;
1508     offchainShares = pending;
1509     proposedOffchainShares = 0;
1510     emit ConfirmOffchainShares(_msgSender(), pending, previous);
1511 }
```

Status

✓ Fixed



[WP-M5] `cancelRedeem()` and `revertRedeemToPending()` trigger a precipitous drop in `sharesPerToken`.

Medium

Issue Description

You must immediately invoke `proposeOffchainShares()` and `confirmOffchainShares()` to update `offchainShares` and maintain consistency; otherwise, `sharesPerToken` will be artificially deflated.

The step sequence documentation fails to mention the `cancelRedeem()` and `revertRedeemToPending()` calling.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1069-L1116>

```

1069 @@ 1069,1073 @@
1074 function cancelRedeem(uint256 _len) public onlyRole(MAINTAINER_ROLE) {
1075     _len = _validateQueueProcessing(redeemQueue.length(), _len);
1076
1077     while (_len > 0) {
1078         bytes memory data = redeemQueue.popFront();
1079         (
1080             address sender,
1081             address receiver,
1082             uint256 shareAmount, // Full share amount
1083             ,
1084             ,
1085             ,
1086             // redeemAssetAmt - not needed
1087             // feeAssetAmt - not needed
1088             // requestTimestamp - not needed
1089             bytes32 id
1090         ) = _decodeRedeemData(data);
1091
1092         redeemInfo[receiver] -= shareAmount;
1093         totalRedeemQueueShares -= shareAmount;
1094         delete snapshotRatios[id];
1095
1096 @@ 1096,1114 @@
1115     }
1116 }

```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1151-L1210>

```

1151 @@ 1151,1158 @@
1159 function revertRedeemToPending(uint256 _len) external onlyRole(OPERATOR_ROLE) {
1160     _len = _validateQueueProcessing(redeemQueue.length(), _len);
1161
1162     // Pop from back of redeem queue and push to front of pending queue
1163     // This maintains order: redeemQueue [a, b] -> pendingQueue [a, b]
1164     for (uint256 i = 0; i < _len; ) {
1165         bytes memory data = redeemQueue.popBack();
1166         (
1167             address sender,
1168             address receiver,
1169             uint256 shareAmount, // Full share amount
1170             ,

```



```
1171 ,
1172 // redeemAssetAmt - discard (was calculated with wrong price)
1173 // feeAssetAmt - discard (was calculated with wrong price)
1174 uint256 requestTimestamp, // Keep original timestamp
1175 bytes32 oldId
1176 ) = _decodeRedeemData(data);
1177
1178 redeemInfo[receiver] -= shareAmount;
1179 totalRedeemQueueShares -= shareAmount;
1180
1181 @@ 1181,1208 @@
1209 }
1210 }
```

```
1585 /**
1586 * @notice Calculate the current shares-per-token ratio in 1e18 precision
1587 * @dev Formula: offchainShares / (totalSupply - totalRedeemQueueShares), scaled
1588 * by 1e18.
1589 * Denominator excludes the final redeem queue (those shares are already
1590 * priced).
1591 * Pending-redeem-queue shares are included in the denominator.
1592 * Returns the 1e18 fallback when either denom is zero (bootstrap / full
1593 * exit) or
1594 * offchainShares has not yet been set by the bot (pre-first-sync). In the
1595 * pre-sync
1596 * window deposits and redeems settle at 1:1 against the underlying asset,
1597 * which is
1598 * economically fair: depositors mint at 1:1 and redeemers get their deposit
1599 * back.
1600 * updateEpoch is a no-op in this window because dailyFee = offchainShares *
1601 * rate = 0.
1602 */
1603 function _sharesPerToken() internal view returns (uint256 ratio) {
1604     uint256 totalSupply = IERC20(address(token)).totalSupply();
1605     uint256 denom = totalSupply - totalRedeemQueueShares;
1606     if (offchainShares == 0 || denom == 0) return 1e18;
1607     ratio = Math.mulDiv(offchainShares, 1e18, denom);
1608 }
```

Status

✓ Fixed



[WP-M6] processDepositQueue uses live token.totalSupply(), producing intra-batch price drift that advantages earlier vs later depositors in the same batch

Medium

Issue Description

`_calculateMintAmount` in `processDepositQueue` resolves the shares-per-token ratio through `_sharesPerToken()`, which reads `token.totalSupply()` **live**. As each iteration of the batch calls `token.mint(...)`, `totalSupply` grows, `_sharesPerToken` decreases, and the **next** depositor in the same batch gets more HYBOND per underlying than the one before.

Although MAINTAINER controls batch ordering, users cannot audit it on-chain, and depositors who happened to queue earlier receive systematically worse pricing than those behind them — a predictable unfairness determined by batch position rather than market state.

`processDepositQueue` mints inside a loop using a ratio derived from live `totalSupply`:

```
666  function processDepositQueue(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
667  _len = _validateQueueProcessing(depositQueue.length(), _len);
668
669  for (uint256 count = 0; count < _len; ) {
670  bytes memory data = depositQueue.front();
671  (
672  address asset,
673  address sender,
674  address receiver,
675  uint256 netAssets,
676  uint256 feeAmt,
677  bytes32 prevId
678  ) = _decodeDepositData(data);
679
680  _validateKyc(sender, receiver);
681
682  depositQueue.popFront();
683  depositInfo[receiver][asset] -= netAssets;
684  unchecked {
685  ++count;
686  }
687
688  uint256 mintedAmount = _calculateMintAmount(asset, netAssets);
```



```
689 token.mint(receiver, mintedAmount);
690
691 emit ProcessDeposit(asset, sender, receiver, netAssets, mintedAmount,
    feeAmt, prevId);
692 }
693 }
```

`_sharesPerToken` reads `totalSupply` live, with no caching or snapshot:

```
1597 function _sharesPerToken() internal view returns (uint256 ratio) {
1598     uint256 totalSupply = IERC20(address(token)).totalSupply();
1599     uint256 denom = totalSupply - totalRedeemQueueShares;
1600     if (offchainShares == 0 || denom == 0) return 1e18;
1601     ratio = Math.mulDiv(offchainShares, 1e18, denom);
1602 }
1603
1604 /**
```

Status

✓ Fixed



[WP-M7] setSnapshotRatio caps input at 1e18, blocking MAINTAINER from replicating legitimate protocol states where `_sharesPerToken() > 1e18`

Medium

Issue Description

`setSnapshotRatio` is documented as a manual override — the fallback when the automatic `snapshotPendingRedeemRatio()` fails or needs correction. Its implementation rejects `_ratio > 1e18` as invalid, but `_sharesPerToken()` itself has no such upper bound and can return values above `1e18` legitimately whenever `offchainShares > (totalSupply - totalRedeemQueueShares)`. This happens in at least two ordinary regimes:

- **Early bootstrap / post-full-exit:** small denominator relative to `offchainShares`.
- **BNY fund appreciation outpacing new deposits / fee minting:** the numerator grows faster than supply.

Because the automatic snapshot accepts ratios `> 1e18` (via `snapshotPendingRedeemRatio`), but the manual override does not, MAINTAINER is unable to replicate the real ratio during an incident. The emergency fallback is forced to use an artificially-low `1e18` ceiling, which **underpays queued redeemers** exactly at the moment when fair pricing matters most.

`setSnapshotRatio` rejects any ratio above `1e18`:

```
1000  /**
1001  * @notice Manually set snapshot ratio for a pending redeem entry
1002  * @dev Fallback for failed auto-snapshots or corrections. Overwrites any existing
1003  *      value.
1004  * @param _id Pending redeem ID
1005  * @param _ratio Shares-per-token ratio in 1e18 precision
1006  */
1007  function setSnapshotRatio(bytes32 _id, uint256 _ratio) external onlyRole(-
1008  MAINTAINER_ROLE) {
1009      if (_ratio == 0 || _ratio > 1e18) revert InvalidInput(_ratio);
1010      snapshotRatios[_id] = _ratio;
1011      emit SetSnapshotRatio(_id, _ratio);
1012  }
```

`_sharesPerToken` has no such cap:



```
1597 function _sharesPerToken() internal view returns (uint256 ratio) {
1598     uint256 totalSupply = IERC20(address(token)).totalSupply();
1599     uint256 denom = totalSupply - totalRedeemQueueShares;
1600     if (offchainShares == 0 || denom == 0) return 1e18;
1601     ratio = Math.mulDiv(offchainShares, 1e18, denom);
1602 }
1603
1604 /**
```

Recommendation

Drop the upper bound. Keep only the zero check:

```
function setSnapshotRatio(bytes32 _id, uint256 _ratio) external onlyRole(MAINTAINER_ROLE)
{
    if (_ratio == 0) revert InvalidInput(_ratio);
    snapshotRatios[_id] = _ratio;
    emit SetSnapshotRatio(_id, _ratio);
}
```

If a sanity bound is desired, pick a number that is clearly above any legitimate protocol state (e.g. $10 \times 1e18$) and document the choice — but do not reuse `1e18` because that's the midpoint of the realistic range, not an upper bound.

Status

✓ Fixed



[WP-M8] PriceOracle.confirmPrice sets round.updatedAt = observedAt, forcing maxStalePeriod to be set wider than any real freshness threshold — defeating the guard it's meant to enforce

Medium

Issue Description

`confirmPrice` runs at T+2 (the on-chain confirmation step), but it writes `round.updatedAt = pendingPriceValue.observedAt` — the **T+0 BNY observation timestamp**, not `block.timestamp`. Consumers read this value through the AggregatorV3 interface and check `block.timestamp - updatedAt` against `Express.maxStalePeriod`.

Because the T+2 confirmation is the normal protocol cadence, `updatedAt` is **always ~2 days in the past at the moment it becomes active**. To let the protocol operate at all, MAINTAINER must set `maxStalePeriod` well above 2 days. But that same `maxStalePeriod` is what's supposed to catch a compromised or abandoned oracle — setting it so wide that a fresh price looks stale also widens the window during which a genuinely stale price passes the check.

The result is a staleness guard whose threshold is dictated by the settlement cadence, not by the freshness policy — it cannot both allow normal operation and catch stale data.

`confirmPrice` stamps `updatedAt` with the T+0 observation time, not the T+2 confirmation time:

```
268 RoundData storage round = roundData[latestRoundValue];
269 round.roundId = latestRoundValue;
270 round.answer = newPrice;
271 round.startedAt = pendingPriceValue.observedAt;
272 round.updatedAt = pendingPriceValue.observedAt;
273 round.answeredInRound = latestRoundValue;
274
275 delete pendingPriceValue;
276
277 emit RoundUpdated(latestRoundValue);
278 }
```

`Express.getPrice` checks freshness against `updatedAt`, inheriting the T+2 offset:



```
630
631 (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound)
    = priceOracle.latestRoundData();
632
633 // Validate price is positive
634 if (answer <= 0) revert InvalidPrice(answer);
635
636 // Check for stale price data
637 if (maxStalePeriod > 0 && block.timestamp - updatedAt > maxStalePeriod) {
638     revert StalePriceData(updatedAt, block.timestamp, maxStalePeriod);
639 }
640
```

Recommendation

Separate the two concepts:

- **round.updatedAt = block.timestamp** — the on-chain write time, used for staleness checks. This is the value all **AggregatorV3** consumers expect.
- Keep **observedAt** as a separate field (it already exists) for audit trail / downstream reporting.

```
// PriceOracle.sol::confirmPrice
round.startedAt = pendingPriceValue.observedAt; // unchanged - audit timestamp
round.updatedAt = block.timestamp;              // <- use on-chain write time
```

After this change, **maxStalePeriod** can be set to a tight value (e.g. a few hours above the expected confirmation cadence) and does what it's supposed to do: catch oracles that have not been confirmed on-chain recently.

Status

✓ Fixed



[WP-M9] Redeem fee is priced at pending → final migration time, not at user-request time — MAINTAINER-initiated `updateRedeemFeeRate` retroactively changes the fee queued users will pay

Medium

Issue Description

`requestRedeem` (T+0) places an entry in `pendingRedeemQueue` **without** locking in a fee. `_processSinglePendingRedeem` (T+2) then calls `txsFee(redeemAssetAmt, TxType.REDEEM)` — which reads the **current** `redeemFeeRate` — and that fee is what the user pays.

`updateRedeemFeeRate` is a MAINTAINER-only setter with no timelock and no grace period for in-flight requests. Users who submitted a redeem request under one published fee schedule can be forced to pay a higher fee retroactively, simply because MAINTAINER adjusted the rate between their request and its settlement.

Impact is modest as long as MAINTAINER is trusted and gives notice, but there is no on-chain mechanism enforcing "fee at time of request" semantics — which is the natural user expectation.

`_processSinglePendingRedeem` reads the live `redeemFeeRate` (via `txsFee`) at pending → final migration:

```
893 pendingRedeemInfo[receiver] -= shareAmount;
894
895 // Price redeem proceeds using the snapshotted ratio
896 uint256 redeemAssetAmt = _redeemAssetAmount(shareAmount, currentRatio,
currentPrice);
897
898 // Calculate fee in redeemAsset (not in token!)
899 uint256 feeAssetAmt = txsFee(redeemAssetAmt, TxType.REDEEM);
900
901 // Add to final redeem queue with asset fee
```

`updateRedeemFeeRate` has no timelock / no grandfathering:



```
499  */
500  function updateRedeemFeeRate(uint256 _rate) external onlyRole(MAINTAINER_ROLE) {
501    if (_rate >= BPS_BASE) revert InvalidInput(_rate);
502    redeemFeeRate = _rate;
503    emit UpdateRedeemFeeRate(_rate);
504  }
```

Recommendation

Pick one of:

- **Lock at request time.** Store `feeRateAtRequest` in the pending queue entry; `_processSinglePendingRedeem` computes `feeAssetAmt` using that stored rate, not the current one.
- **Timelock.** Require `updateRedeemFeeRate` increases to take effect after a delay, so users can cancel in-flight redemptions before the new rate applies.

The first is cleaner and self-documenting on-chain; the second is less invasive.

Status

📌 Acknowledged



[WP-M10] `Express.updateEpoch()` fails to account for the price discrepancy between off-chain BNY shares and Hybond token.

Medium

Issue Description

At L1440, `Express.updateEpoch()` treats `dailyFee` as the quantity of off-chain BNY shares.

However, at L1443 and L1444, `Express.updateEpoch()` incorrectly uses `dailyFee` as the quantity of Hybond `token` shares.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1425-L1449>



```
1425  /**
1426  * @notice Update epoch and accrue management fees based on offchainShares
1427  * @dev Fee is charged on the offchain BNY value (the real AUM), not on circulating
1428  *       HYBOND.
1429  *       This matches how a real-world fund charges mgmt fees on AUM rather than
1430  *       on issued shares.
1431  *       Can only be called after timeBuffer has elapsed since last update. Reverts
1432  *       PendingProposalExists if an offchain-shares proposal is unconfirmed –
1433  *       running with
1434  *       stale offchainShares would bake in wrong dilution.
1435  */
1436  function updateEpoch() external onlyRole(OPERATOR_ROLE) {
1437      if (mgtFeeRate == 0) revert MgtFeeDisabled();
1438      if (proposedOffchainShares != 0) revert PendingProposalExists-
1439      (proposedOffchainShares);
1440      if (lastUpdateTS != 0 && block.timestamp < lastUpdateTS + timeBuffer) {
1441          revert UpdateTooEarly(block.timestamp);
1442      }
1443      uint256 dailyFee = _trim(_calculateDailyMgtFee(offchainShares));
1444      if (dailyFee > 0) {
1445          if (mgtFeeTo == address(0)) revert InvalidAddress();
1446          totalMgtFeeUnclaimed += dailyFee;
1447          token.mint(mgtFeeTo, dailyFee);
1448      }
1449      lastUpdateTS = block.timestamp;
1450      emit UpdateEpoch(dailyFee, offchainShares);
1451  }
```

```
1576  /**
1577  * @notice Calculate daily management fee from a base amount (typically
1578  *       offchainShares)
1579  * @param _base The AUM base to charge the annualized rate against
1580  * @return fee Daily management fee amount
1581  */
1582  function _calculateDailyMgtFee(uint256 _base) internal view returns (uint256 fee)
1583  {
1584      fee = Math.mulDiv(_base, mgtFeeRate, DAYS_IN_YEAR * BPS_BASE);
1585  }
```

```
766  /**
767  * @notice Queue a redeem request for T+2 pricing
768  * @dev Request goes to pending queue without price calculation
769  *       Fees held in contract until processing (refundable on cancellation)
```



```
770 *
771 *     For the mgtFeeTo wallet, the caller-supplied _shareAmount is IGNORED and
772 *     the full live fee balance (totalMgtFeeUnclaimed) is redeemed instead.
773 *     This prevents fee-share provenance desync when the mgtFeeTo wallet holds
774 *     any non-fee shares (e.g. from direct transfers). The redeemMinimum check
775 *     is
776 *     skipped for mgtFeeTo so that small daily fee balances can still be claimed.
777 * @param _to The address to receive redeemed asset
778 * @param _shareAmount The amount of share to redeem (IGNORED if caller is mgtFeeTo)
779 */
780 function requestRedeem(address _to, uint256 _shareAmount) external
whenNotPausedRedeem {
781     address from = _msgSender();
782     _validateKyc(from, _to);
783     if (from == mgtFeeTo) {
784         if (totalMgtFeeUnclaimed == 0) revert InvalidAmount();
785         _shareAmount = totalMgtFeeUnclaimed;
786     } else {
787         if (_shareAmount < redeemMinimum) {
788             revert RedeemLessThanMinimum(_shareAmount, redeemMinimum);
789         }
790     }
791
792     // Collect full token amount to contract
793     // No fee deduction - fee will be charged from redeemAsset output
794     IERC20(address(token)).safeTransferFrom(from, address(this), _shareAmount);
795
796     // Track full token amount
797     pendingRedeemInfo[_to] += _shareAmount;
798
799     bytes32 id = keccak256(abi.encode(from, _to, _shareAmount, block.timestamp,
800 _nonce++));
801
802     bytes memory data = abi.encode(
803         from,
804         _to,
805         _shareAmount, // Full share amount
806         block.timestamp, // Store request time for T+2 check
807         id
808     );
809     pendingRedeemQueue.pushBack(data);
810 }
```



```
811     emit AddToPendingRedeemQueue(from, _to, _shareAmount, id);
812 }
```

```
865  /**
866   * @notice Internal helper to process a single pending redeem
867   * @param currentPrice The current T+2 price to apply
868   * @return success True if redeem was processed, false if not ready yet
869   */
870   function _processSinglePendingRedeem(uint256 currentPrice) internal returns (bool
success) {
871     bytes memory data = pendingRedeemQueue.front();
872     (
873     address sender,
874     address receiver,
875     uint256 shareAmount, // Full share amount (no fee deduction)
876     uint256 requestTimestamp,
877     bytes32 pendingId
878     ) = _decodePendingRedeemData(data);
879
880     // Check if convertRedeemRequestsDelay has elapsed
881     if (block.timestamp < requestTimestamp + convertRedeemRequestsDelay) {
882         return false;
883     }
884
885     // Ratio must be snapshotted before processing
886     uint256 currentRatio = snapshotRatios[pendingId];
887     if (currentRatio == 0) revert RatioNotSnapshotted(pendingId);
888
889     _validateKyc(sender, receiver);
890
891     // Remove from pending queue and migrate snapshot to final ID
892     pendingRedeemQueue.popFront();
893     pendingRedeemInfo[receiver] -= shareAmount;
894
895     // Price redeem proceeds using the snapshotted ratio
896     uint256 redeemAssetAmt = _redeemAssetAmount(shareAmount, currentRatio,
currentPrice);
897
898     @@ 898,933 @@
934 }
```



```
1604  /**
1605  * @notice Convert token amount into redeem asset amount using a ratio and price
1606  * snapshot
1607  * @param _shareAmount Full token amount being redeemed
1608  * @param _ratio Shares-per-token ratio in 1e18 precision
1609  * @param _price Price snapshot in 1e18 precision
1610  */
1611  function _redeemAssetAmount(
1612    uint256 _shareAmount,
1613    uint256 _ratio,
1614    uint256 _price
1615  ) internal view returns (uint256 redeemAssetAmt) {
1616    uint256 backedShareAmount = Math.mulDiv(_shareAmount, _ratio, 1e18);
1617    redeemAssetAmt = _trimAsset(
1618      Math.mulDiv(convertToUnderlying(redeemAsset, backedShareAmount), _price,
1619        1e18),
1620      redeemAsset
1621    );
1622  }
```

Status

✓ Fixed



[WP-M11] When the pendingRedeemQueue is non-empty, the offchainShares used to accrue the DailyMgtFee may be artificially inflated, as it includes off-chain BNY shares that have already been marked for sale.

Medium

Issue Description

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/docs/management-fee.md>

```
1  # Overview
2
3  Hybond is a token issued via `Token.sol`, used to tokenize real-world assets: a BNY
  ETF Fund. The Express contract manages deposits, redeems, and the daily management
  fee.
4
5  ## Key state
6
7  @@ 7,10 @@
11
12  ## sharesPerToken formula
13
14  ```
15  sharesPerToken = offchainShares / (totalSupply - totalRedeemQueueShares)
16  ```
17
18  - Denominator excludes the final redeem queue (those shares are locked at their snapshot
  ratio).
19
20  - Returns the `1e18` fallback when either the denominator is zero (bootstrap / full
  exit) or `offchainShares == 0` (pre-sync). In the pre-sync window, deposits and redeems
  settle at 1:1 – economically fair (users get the asset-value of their shares) and safe
  because `updateEpoch` is a no-op when `offchainShares == 0`.
20
21  ## Roles
22
23  - `OPERATOR_ROLE` – processes queues, proposes `offchainShares`, calls `updateEpoch`.
24  - `CONFIRM_ROLE` – echo-confirms the proposed `offchainShares` value.
25  - `MAINTAINER_ROLE` – config updates, `processDepositQueue`.
26
27  ## Token Mint Workflow (`Express.sol::requestDeposit`)
28
29  1. User calls `requestDeposit` to deposit USDC to the contract at T+0.
30  2. Fund operator transfers the USDC to buy BNY ETF Fund off-chain at T+0 EOD.
```



```
31 3. At T+2, BNY emails us the share price and share amount of the BNY ETF Fund.
32 4. Fund operator updates the share price in `PriceOracle.sol` via `proposePrice` and
   `confirmPrice`.
33 5. Fund operator calls `processDepositQueue` (MAINTAINER_ROLE) to mint Hybond tokens
   to users.
34 6. Fund operator calls `proposeOffchainShares` (OPERATOR_ROLE) and CONFIRM_ROLE calls
   `confirmOffchainShares` to sync the new BNY share balance.
35 7. User receives Hybond tokens in their wallet.
36
37 ## Token Redeem Workflow (`Express.sol::requestRedeem`)
38
39 1. User calls `requestRedeem` to redeem Hybond for USDC at T+0.
40 2. Fund operator reads `sharesPerToken` to calculate the off-chain BNY shares to sell.
41 3. Fund operator sells BNY ETF Fund shares off-chain at T+0 EOD.
42 4. At T+2, BNY emails us the sell price and USDC amount we will receive.
43 5. Fund operator updates the price in `PriceOracle.sol` via `proposePrice` and
   `confirmPrice`.
44 6. Fund operator calls `processPendingRedeems` (OPERATOR_ROLE) to lock in USDC
   amounts using the current price and `sharesPerToken` ratio. Requests move from
   `pendingRedeemQueue` to `redeemQueue`.
45 7. At T+4, we receive the USDC from BNY.
46 8. Fund operator calls `processRedeemQueue` (OPERATOR_ROLE) to burn Hybond tokens and
   transfer USDC to users (FIFO).
47
48 ## Management Fee
49
50 1. Fund operator calls `updateEpoch` (OPERATOR_ROLE) daily to calculate and mint that
   day's management fee to `mgtFeeTo`.
51 2. Fee base is `offchainShares` (the real AUM), not circulating HYBOND. Matches how
   real-world funds charge management fees on AUM rather than on issued shares.
52 3. Formula: `dailyFee = offchainShares * mgtFeeRate / (365 * BPS_BASE)`, trimmed to
   `trimDecimals` precision.
53 4. `updateEpoch` reverts when `mgtFeeRate == 0` (`MgtFeeDisabled`), when a proposal is
   pending and unconfirmed (`PendingProposalExists`), or when `timeBuffer` hasn't elapsed
   since the last call (`UpdateTooEarly`).
54 5. The `PendingProposalExists` guard forces the bot to confirm `offchainShares` before
   accruing fees – prevents fee mint against stale AUM.
55
56 ## Combined Daily Workflow
57
58 @@ 58,70 @@
59
60
61
62 ### 4. Fund Operator Side (T+2, Daily Batch)
63
64 Each step has its own trigger – the ordering below is the required sequence **when a
   step runs**, not a requirement that every step runs every day.
65
```

76	Step	Function	Role	When to	Why this
	run				
	order / notes				
77	----	-----	-----		
78	4.1	<code>`proposePrice`</code> (BNY publishes price) before any mint/redeem arithmetic	OPERATOR_ROLE	**Business days only** Oracle price must be fresh	
79	4.2	<code>`confirmPrice`</code> (pairs with 4.1) price	CONFIRM_ROLE	**Business days only** Echo-confirm the proposed	
80	4.3	<code>`processDepositQueue`</code> deposit requests are ready** (deposits made 2 business days ago)	MAINTAINER_ROLE	**Only if T+2	Mint priced against
		previous-day's <code>`offchainShares`</code> and fresh oracle price			
81	4.4	<code>`processPendingRedeems`</code> requests are ready**	OPERATOR_ROLE	**Only if T+2 redeem	Move delay-ripe entries into
		the final queue at their ~2-day-old snapshot ratio. Requires oracle price; independent			
		of today's <code>`offchainShares`</code> sync and fee mint			
82	4.5	<code>`proposeOffchainShares`</code> changed** (claimed mgt fee, buys, sells), **T+2**	OPERATOR_ROLE	**Only when BNY balance	Sync new AUM before any fee
		accrual or new snapshot			
83	4.6	<code>`confirmOffchainShares`</code> run** (pairs with it) <code>`proposedOffchainShares`</code>	CONFIRM_ROLE	**Only when 4.5 was	Echo-confirm; clears
84	4.7	<code>`updateEpoch`</code> **Daily**	OPERATOR_ROLE		Accrue
		daily fee on <code>`offchainShares`</code> . Reverts if a proposal from 4.5 is unconfirmed			
85	4.8	<code>`processRedeemQueue`</code> arrived** for that batch (FIFO, liquidity-gated)	OPERATOR_ROLE	**Only if T+4 USDC has	Burn Hybond and pay USDC
86	4.9	<code>`snapshotPendingRedeemRatio`</code> fresh redeem requests today** (new pending entries) for today's fresh pending entries (consumed ~2 days later)	OPERATOR_ROLE	**Only if there were	Freeze end-of-day ratio
87					
88	@@ 88,139 @@				

As described in the "T+0 EOD" documentation, BNY ETF Fund shares are sold off-chain. The `_sharesPerToken()` for the corresponding `pendingId` is fixed via `snapshotPendingRedeemRatio()`, effectively locking the value of these off-chain shares.

However, these off-chain shares may not be deducted from `offchainShares` until T+2. If they were deducted earlier, `_sharesPerToken()` would be erroneously low because the corresponding on-chain shares are not added to `totalRedeemQueueShares` until T+2. Consequently, between "T+0 EOD" and T+2, `offchainShares` may deviate from "the real AUM," assuming "the real AUM" excludes the BNY ETF Fund shares already sold at "T+0 EOD."

`updateEpoch()` continues to charge the `DailyMgtFee` on the entire `offchainShares` balance, including the portion locked since "T+0 EOD." This forces holders of the remaining off-chain



shares—those not part of the fixed `pendingRedeemQueue`—to bear a management fee higher than the intended `mgtFeeRate`.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1425-L1449>

```
1425  /**
1426  * @notice Update epoch and accrue management fees based on offchainShares
1427  * @dev Fee is charged on the offchain BNY value (the real AUM), not on circulating
1428  *       HYBOND.
1429  *       This matches how a real-world fund charges mgmt fees on AUM rather than
1430  *       on issued shares.
1431  *       Can only be called after timeBuffer has elapsed since last update. Reverts
1432  *       PendingProposalExists if an offchain-shares proposal is unconfirmed –
1433  *       running with
1434  *       stale offchainShares would bake in wrong dilution.
1435  */
1436  function updateEpoch() external onlyRole(OPERATOR_ROLE) {
1437      if (mgtFeeRate == 0) revert MgtFeeDisabled();
1438      if (proposedOffchainShares != 0) revert PendingProposalExists-
1439      (proposedOffchainShares);
1440      if (lastUpdateTS != 0 && block.timestamp < lastUpdateTS + timeBuffer) {
1441          revert UpdateTooEarly(block.timestamp);
1442      }
1443      uint256 dailyFee = _trim(_calculateDailyMgtFee(offchainShares));
1444      if (dailyFee > 0) {
1445          if (mgtFeeTo == address(0)) revert InvalidAddress();
1446          totalMgtFeeUnclaimed += dailyFee;
1447          token.mint(mgtFeeTo, dailyFee);
1448      }
1449      lastUpdateTS = block.timestamp;
1450      emit UpdateEpoch(dailyFee, offchainShares);
1451  }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1576-L1583>



```
1576  /**
1577  * @notice Calculate daily management fee from a base amount (typically
1578  * @param _base The AUM base to charge the annualized rate against
1579  * @return fee Daily management fee amount
1580  */
1581  function _calculateDailyMgtFee(uint256 _base) internal view returns (uint256 fee)
1582  {
1583      fee = Math.mulDiv(_base, mgtFeeRate, DAYS_IN_YEAR * BPS_BASE);
1584  }
```

```
969  /**
970  * @notice Snapshot current sharesPerToken ratio into pending redeem entries within
971  * @dev Called daily at 4pm SGT after the daily batch (processPendingRedeems,
972  *      updateEpoch). Operator passes explicit indices so a large queue can be
973  *      chunked (gas budgeting) or partially re-snapshotted to recover from a
974  *      botched run.
975  *      Existing non-zero ratios within the range are skipped (idempotent overwrite
976  *      is via setSnapshotRatio). Emits SnapshotPendingRedeemRatio for every id newly
977  *      written.
978  * @param _start Inclusive start index in pendingRedeemQueue
979  * @param _end Exclusive end index in pendingRedeemQueue (must satisfy _start <
980  *      _end <= queueLen)
981  */
982  function snapshotPendingRedeemRatio(uint256 _start, uint256 _end) external
983  onlyRole(OPERATOR_ROLE) {
984      uint256 queueLen = pendingRedeemQueue.length();
985      if (queueLen == 0) revert EmptyQueue();
986      if (_end > queueLen || _start >= _end) revert InvalidInput(_end);
987
988      uint256 currentRatio = _sharesPerToken();
989
990      for (uint256 i = _start; i < _end; ) {
991          (, , , , bytes32 id) = _decodePendingRedeemData(pendingRedeemQueue.at(i));
992          if (snapshotRatios[id] == 0) {
993              snapshotRatios[id] = currentRatio;
994              emit SnapshotPendingRedeemRatio(id, currentRatio);
995          }
996          ++i;
997      }
998  }
```

```

996 }
997 }
998 }

```

```

842 /**
843  * @notice Process pending redeems (T+N discipline enforced by operator off-chain)
844  * @dev Uses snapshotted ratio from snapshotPendingRedeemRatio(). Reverts if ratio
      not set.
845  *      Operator must pass an explicit positive _len; _len = 0 makes the loop a
      no-op
846  *      and reverts with NoPendingRedeemsReady (process-all sentinel is not supported
      here).
847  * @param _len Number of pending requests to process (must be > 0)
848  */
849  function processPendingRedeems(uint256 _len) external onlyRole(OPERATOR_ROLE) {
850      uint256 processed;
851      uint256 currentPrice = getPrice();
852
853      while (processed < _len && !pendingRedeemQueue.empty()) {
854          if (!_processSinglePendingRedeem(currentPrice)) {
855              break;
856          }
857          unchecked {
858              ++processed;
859          }
860      }
861
862      if (processed == 0) revert NoPendingRedeemsReady();
863  }

```

```

865 /**
866  * @notice Internal helper to process a single pending redeem
867  * @param currentPrice The current T+2 price to apply
868  * @return success True if redeem was processed, false if not ready yet
869  */
870  function _processSinglePendingRedeem(uint256 currentPrice) internal returns (bool
      success) {
871      bytes memory data = pendingRedeemQueue.front();
872      (
873          address sender,
874          address receiver,
875          uint256 shareAmount, // Full share amount (no fee deduction)
876          uint256 requestTimestamp,
877          bytes32 pendingId

```



```
878 ) = _decodePendingRedeemData(data);
879
880 // Check if convertRedeemRequestsDelay has elapsed
881 if (block.timestamp < requestTimestamp + convertRedeemRequestsDelay) {
882     return false;
883 }
884
885 // Ratio must be snapshotted before processing
886 uint256 currentRatio = snapshotRatios[pendingId];
887 if (currentRatio == 0) revert RatioNotSnapshotted(pendingId);
888
889 _validateKyc(sender, receiver);
890
891 // Remove from pending queue and migrate snapshot to final ID
892 pendingRedeemQueue.popFront();
893 pendingRedeemInfo[receiver] -= shareAmount;
894
895 // Price redeem proceeds using the snapshotted ratio
896 uint256 redeemAssetAmt = _redeemAssetAmount(shareAmount, currentRatio,
currentPrice);
897
898 @@ 898,921 @@
922
923 redeemInfo[receiver] += shareAmount;
924 totalRedeemQueueShares += shareAmount;
925
926 @@ 926,933 @@
934 }
```



```
1604  /**
1605  * @notice Convert token amount into redeem asset amount using a ratio and price
1606  * snapshot
1607  * @param _shareAmount Full token amount being redeemed
1608  * @param _ratio Shares-per-token ratio in 1e18 precision
1609  * @param _price Price snapshot in 1e18 precision
1610  */
1611  function _redeemAssetAmount(
1612    uint256 _shareAmount,
1613    uint256 _ratio,
1614    uint256 _price
1615  ) internal view returns (uint256 redeemAssetAmt) {
1616    uint256 backedShareAmount = Math.mulDiv(_shareAmount, _ratio, 1e18);
1617    redeemAssetAmt = _trimAsset(
1618      Math.mulDiv(convertToUnderlying(redeemAsset, backedShareAmount), _price,
1619        1e18),
1620      redeemAsset
1621    );
1622  }
```

```
1585  @@ 1585,1596 @@
1597  function _sharesPerToken() internal view returns (uint256 ratio) {
1598    uint256 totalSupply = IERC20(address(token)).totalSupply();
1599    uint256 denom = totalSupply - totalRedeemQueueShares;
1600    if (offchainShares == 0 || denom == 0) return 1e18;
1601    ratio = Math.mulDiv(offchainShares, 1e18, denom);
1602  }
```

Status

✓ Fixed



[WP-M12] Management Fee Accrual Depends on Call Frequency Instead of Elapsed Time

Medium

Issue Description

`Express.updateEpoch()` accrues a full day of management fees every time it is called, but the accrued amount is not scaled by the real time elapsed since the last epoch update. This breaks the invariant that management fees should track wall-clock time. In the worst case, if `timeBuffer` is set to `0`, an operator can accrue arbitrary numbers of "daily" fees back-to-back and then mint them via `claimMgtFee()`, causing unbounded dilution of holders.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L292-L324>

```
292  function initialize(  
293  address _token,  
294  address _redeemAsset,  
295  address _treasury,  
296  address _txFeeTo,  
297  address _mgtFeeTo,  
298  address admin,  
299  address _assetRegistry,  
300  address _priceOracle,  
301  DepositRedeemLimiterCfg memory cfg  
302  ) external initializer {  
303  @@ 303,315 @@  
316  token = IToken(_token);  
317  redeemAsset = _redeemAsset;  
318  treasury = _treasury;  
319  txFeeTo = _txFeeTo;  
320  mgtFeeTo = _mgtFeeTo;  
321  assetRegistry = IAssetRegistry(_assetRegistry);  
322  priceOracle = IPriceFeed(_priceOracle);  
323  
324  __DepositRedeemLimiter_init(cfg.depositMinimum, cfg.redeemMinimum,  
    cfg.firstDepositAmount);
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L1358-L1405>

```

1358 function updateEpoch(uint256 _circulatingSupply) external onlyRole(OPERATOR_ROLE)
1359 {
1360     if (lastUpdateTS != 0) {
1361         if (block.timestamp < lastUpdateTS + timeBuffer) {
1362             revert UpdateTooEarly(block.timestamp);
1363         }
1364     }
1365     epoch++;
1366
1367     @@ 1367,1376 @@
1377     uint256 dailyFee = _trim(_calculateDailyMgtFee(circulating));
1378     unclaimedMgtFee += dailyFee;
1379
1380     lastUpdateTS = block.timestamp;
1381     emit UpdateEpoch(unclaimedMgtFee, dailyFee, epoch, circulating);
1382 }
1383
1384 @@ 1384,1391 @@
1392 unclaimedMgtFee -= _amount;
1393 token.mint(mgtFeeTo, _amount);
1394
1395 emit ClaimMgtFee(mgtFeeTo, _amount);
1396 }
1397
1398 @@ 1398,1401 @@
1402 function updateTimeBuffer(uint256 _timeBuffer) external onlyRole(MAINTAINER_ROLE)
1403 {
1404     timeBuffer = _timeBuffer;
1405     emit UpdateTimeBuffer(_timeBuffer);
1406 }
1407
1485 function _calculateDailyMgtFee(uint256 _circulatingSupply) internal view returns
1486 (uint256 fee) {
1487     fee = Math.mulDiv(_circulatingSupply, mgtFeeRate, DAYS_IN_YEAR * BPS_BASE);
1488 }

```

PoC

1. Set a non-zero `mgtFeeRate` and ensure the token has a non-zero circulating supply.
2. Set `timeBuffer` to 0, or to a value below one day such as 20 hours.
3. Call `updateEpoch()` multiple times.



4. Observe that each call adds the same full-day fee amount even if almost no time has passed.
5. Call `claimMgtFee()` to mint the over-accrued fee to `mgtFeeTo`.

```
// Pseudocode
express.updateMgtFeeRate(300); // 3% annual fee
express.updateTimeBuffer(0);

express.updateEpoch(0);
uint256 feeAfterFirstCall = express.unclaimedMgtFee();

express.updateEpoch(0);
uint256 feeAfterSecondCall = express.unclaimedMgtFee();

assert(feeAfterSecondCall == feeAfterFirstCall * 2);
```

Recommendation

Accrue management fees proportionally to elapsed time instead of call count, and enforce a sane non-zero minimum epoch interval.

```
uint256 elapsed = lastUpdateTS == 0 ? 1 days : block.timestamp - lastUpdateTS;
if (elapsed < MIN_EPOCH_INTERVAL) revert UpdateTooEarly(block.timestamp);

uint256 fee = Math.mulDiv(circulating, mgtFeeRate * elapsed, 365 days * BPS_BASE);
unclaimedMgtFee += _trim(fee);
lastUpdateTS = block.timestamp;
```

`initialize()` should also set a safe default `timeBuffer`, and `updateTimeBuffer()` should reject 0.

Status

📌 Acknowledged



[WP-M13] Deposit Queue Head-Blocking DOS

Medium

Issue Description

`processDepositQueue()` reads the front item, re-checks KYC, calls `convertFromUnderlying()`, `getPrice()`, and `token.mint()` before popping. If the front entry becomes unprocessable (KYC revoked, asset removed from registry, stale price, issueCap exceeded, receiver banned), the entire batch reverts and FIFO processing cannot advance past that item. The only escape is `cancelDeposit()`, but that function requires Express to be pre-funded with `netAssets + feeAmt` because the original deposit already forwarded fees to `txFeeTo` and net to `treasury`. One bad deposit therefore blocks all subsequent deposits indefinitely.

PoC

1. A KYC'd user calls `requestDeposit()` with a valid asset and amount. Fees go to `txFeeTo`, net goes to `treasury`.
2. Before the maintainer processes the queue, the user's KYC is revoked (or the deposit asset is removed from AssetRegistry, or the oracle goes stale, or `issueCap` is reached).
3. Maintainer calls `processDepositQueue(n)`. The loop hits the front item, `_validateKyc()` reverts, and the entire transaction fails.
4. All deposits behind this entry are blocked.
5. To unblock, maintainer must call `cancelDeposit()`, but Express holds no funds. Treasury and `txFeeTo` must first send funds back to Express.



```
// Step 1: User deposits
express.requestDeposit(usdc, 1000e6, user); // funds leave Express immediately

// Step 2: KYC revoked
express.revokeKycInBulk([user]);

// Step 3: Queue blocked
express.processDepositQueue(0); // reverts: NotInKycList

// Step 4: Cancel requires pre-funding
// treasury must send funds back to Express first
usdc.transfer(address(express), 1000e6); // manual recovery
express.cancelDeposit(1);
```

Vulnerable Code

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L576-L606>

```
576  function processDepositQueue(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
577  _len = _validateQueueProcessing(depositQueue.length(), _len);
578
579  @@ 579,588 @@
589
590  _validateKyc(sender, receiver);
591
592  @@ 592,596 @@
597
598  uint256 amount = convertFromUnderlying(asset, netAssets);
599  uint256 price = getPrice();
600  uint256 mintedAmount = _trim(Math.mulDiv(amount, 1e18, price));
601
602  token.mint(receiver, mintedAmount);
603
604  emit ProcessDeposit(asset, sender, receiver, netAssets, mintedAmount,
605  feeAmt, prevId);
606  }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L471-L502>



```
471 function requestDeposit(address _asset, uint256 _amount, address _receiver) external  
    whenNotPausedDeposit {  
472 @@ 472,486 @@  
487     uint256 feeAmt = txsFee(_amount, TxType.DEPOSIT);  
488     uint256 netAmt = _amount - feeAmt;  
489  
490     if (feeAmt > 0) {  
491         IERC20(_asset).safeTransferFrom(sender, txFeeTo, feeAmt);  
492     }  
493     IERC20(_asset).safeTransferFrom(sender, treasury, netAmt);  
494  
495     depositInfo[_receiver] += netAmt;  
496 @@ 496,496 @@  
497     bytes32 id = keccak256(abi.encode(_asset, sender, _receiver, netAmt, feeAmt,  
        block.timestamp, _nonce++));  
498     bytes memory data = abi.encode(_asset, sender, _receiver, netAmt, feeAmt, id);  
499     depositQueue.pushBack(data);  
500  
501     emit AddToDepositQueue(_asset, sender, _receiver, netAmt, feeAmt, id);  
502 }
```

Recommendation

Add a skip/quarantine mechanism for unprocessable entries, or use try/catch per item to isolate failures. Alternatively, retain deposited funds in Express until processing so that cancellation does not require external pre-funding.

Status

✓ Fixed



[WP-M14] PriceOracle Propose/Confirm Not Bound to Specific Value

Medium

Issue Description

`proposePrice()` does not check whether a pending proposal already exists. A second call silently overwrites `pendingPriceValue`. `confirmPrice()` confirms whatever is currently pending without binding the confirmer to a specific reviewed value. This weakens the intended 2-step approval model because the operator can replace a reviewed proposal with another still-within-guardrails value right before confirmation, and the confirmer would unknowingly approve a different price.

PoC

1. Operator proposes price `1.05e8`. Confirmer reviews and prepares to confirm.
2. Before confirmer's tx lands, operator front-runs with `proposePrice(1.10e8)` (still within deviation bounds).
3. Confirmer's `confirmPrice()` tx executes and confirms `1.10e8` instead of the reviewed `1.05e8`.

```
// Operator proposes 1.05
oracle.proposePrice(1.05e8);

// Confirmer reviews and submits confirmPrice()
// Meanwhile, operator front-runs:
oracle.proposePrice(1.10e8); // overwrites silently

// Confirmer's tx lands
oracle.confirmPrice(); // confirms 1.10e8, not the reviewed 1.05e8
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/PriceOracle.sol#L206-L228>



```
206     function proposePrice(uint256 _price) external onlyRole(OPERATOR_ROLE) {
207     @@ 207,218 @@
219
220     pendingPriceValue = PendingPrice({
221     value: _price,
222     proposedAt: block.timestamp,
223     proposer: msg.sender,
224     exists: true
225     });
226
227     emit PriceProposed/latestPrice, _price, msg.sender);
228 }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/PriceOracle.sol#L230-L256>

```
230     function confirmPrice() external onlyRole(CONFIRMER_ROLE) {
231     if (!pendingPriceValue.exists) revert PendingPriceNotSet();
232     @@ 232,236 @@
237     uint256 newPrice = pendingPriceValue.value;
238     @@ 238,239 @@
```

Recommendation

Reject `proposePrice()` while a pending value exists (the contract already has `PendingPriceExists` error but doesn't use it in `proposePrice`), or require `confirmPrice(uint256 expectedPrice)` so confirmation is bound to the reviewed proposal.

```
function proposePrice(uint256 _price) external onlyRole(OPERATOR_ROLE) {
    if (_price == 0) revert InvalidPrice();
+    if (pendingPriceValue.exists) revert PendingPriceExists();
    ...
}
```

Status

✓ Fixed



[WP-M15] cancelDeposit is intended to refund the fee, but the fee has already been transferred to txFeeTo during requestDeposit.

Medium

Issue Description

```
647 function cancelDeposit(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
648     _len = _validateQueueProcessing(depositQueue.length(), _len);
649
650     while (_len > 0) {
651         bytes memory data = depositQueue.front();
652         (
653             address asset,
654             address sender,
655             address receiver,
656             uint256 netAssets,
657             uint256 feeAmt,
658             bytes32 prevId
659         ) = _decodeDepositData(data);
660
661         depositQueue.popFront();
662         depositInfo[receiver][asset] -= netAssets;
663
664         unchecked {
665             --_len;
666         }
667
668         uint256 refundAmt = netAssets + feeAmt;
669         depositEscrowBalance[sender][asset] += refundAmt;
670         emit DepositEscrowIn(sender, asset, refundAmt);
671
672         emit CancelProcessDeposit(asset, sender, receiver, netAssets, feeAmt,
673             prevId);
674     }
```

```
504 function requestDeposit(address _asset, uint256 _amount, address _receiver) external
whenNotPausedDeposit {
505     address sender = _msgSender();
506     if (_amount == 0) revert InvalidAmount();
507
508     _validateKyc(sender, _receiver);
509 }
```



```
510     uint256 equivalentAmount = convertFromUnderlying(_asset, _amount);
511     if (!firstDeposit[sender]) {
512         if (equivalentAmount < firstDepositAmount) {
513             revert FirstDepositLessThanRequired(equivalentAmount, firstDepositAmount);
514         }
515         firstDeposit[sender] = true;
516     } else if (equivalentAmount < depositMinimum) {
517         revert DepositLessThanMinimum(equivalentAmount, depositMinimum);
518     }
519
520     uint256 feeAmt = txsFee(_amount, TxType.DEPOSIT);
521     uint256 netAmt = _amount - feeAmt;
522
523     if (feeAmt > 0) {
524         IERC20(_asset).safeTransferFrom(sender, txFeeTo, feeAmt);
525     }
526     IERC20(_asset).safeTransferFrom(sender, treasury, netAmt);
527
528     depositInfo[_receiver][_asset] += netAmt;
529
530     bytes32 id = keccak256(abi.encode(_asset, sender, _receiver, netAmt, feeAmt,
531                                     block.timestamp, _nonce++));
532     bytes memory data = abi.encode(_asset, sender, _receiver, netAmt, feeAmt, id);
533     depositQueue.pushBack(data);
534
535     emit AddToDepositQueue(_asset, sender, _receiver, netAmt, feeAmt, id);
536 }
```

Status

📌 Acknowledged



[WP-L16] `updateMgtFeeTo` lacks on-chain enforcement of its documented invariants, silently corrupting `totalMgtFeeUnclaimed` when rotated with in-flight redemptions

Low

Issue Description

`updateMgtFeeTo()` changes the management-fee recipient with no on-chain check that the redeem queues are empty or that `totalMgtFeeUnclaimed == 0`. The function's own NatSpec explicitly documents these as required invariants, and the sibling function `updateRedeemAsset` does enforce a comparable `QueuesNotEmpty` check. `updateMgtFeeTo` does not.

The consequence is a silently corrupted fee tracker:

1. Fee shares minted to the `old mgtFeeTo` sit in `pendingRedeemQueue` after the old recipient requests redemption.
2. MAINTAINER calls `updateMgtFeeTo(new)`. `mgtFeeTo` flips.
3. `_processSinglePendingRedeem` decrements `totalMgtFeeUnclaimed` only when `sender == mgtFeeTo`. The `sender` of the in-flight entries is the `old` recipient, so the decrement is **silently skipped** — no revert, no event.
4. `totalMgtFeeUnclaimed` stays permanently inflated by the unreconciled amount.
5. `circulatingSupply() = totalSupply - totalRedeemQueueShares - totalMgtFeeUnclaimed` now underestimates circulating supply for the life of the contract, which in turn suppresses `sharesPerToken` (i.e. dilutes every subsequent deposit/redeem against every holder).

This is a permanent, unrecoverable accounting drift — no function re-derives `totalMgtFeeUnclaimed` from first principles.

`updateMgtFeeTo` — the NatSpec spells out the required preconditions but the implementation enforces none of them:



```
450  * @dev Requires all live fee tokens to be redeemed and burned first
      (totalMgtFeeUnclaimed == 0).
451  *      Also requires both redeem queues (pendingRedeemQueue, redeemQueue) to be
      empty.
452  *      In-flight entries from the old mgtFeeTo would carry its address as `from`;
      on
453  *      cancel/revert those branches would fail to match the new mgtFeeTo and skip
      the
454  *      totalMgtFeeUnclaimed decrement, corrupting the live fee tracker.
455  * @param _address The new management fee recipient address
456  */
457  function updateMgtFeeTo(address _address) external onlyRole(MAINTAINER_ROLE) {
458      if (_address == address(0)) revert InvalidAddress();
459      mgtFeeTo = _address;
460      emit UpdateMgtFeeTo(_address);
461  }
```

_processSinglePendingRedeem — the decrement that is silently skipped when **sender** (the old **mgtFeeTo**) no longer equals the current **mgtFeeTo**:

```
925  // Decrement live mgt fee counter when fee tokens are priced in.
926  if (sender == mgtFeeTo && totalMgtFeeUnclaimed >= shareAmount) {
927      totalMgtFeeUnclaimed -= shareAmount;
928  }
929
930  emit ProcessPendingRedeem(sender, receiver, shareAmount, currentPrice,
      pendingId, finalId);
```

For comparison, **updateRedeemAsset** applies the exact guard pattern that is missing here:

```
473  if (_address == address(0)) revert InvalidAddress();
474  if (depositQueue.length() != 0 || pendingRedeemQueue.length() != 0 ||
      redeemQueue.length() != 0) {
475      revert QueuesNotEmpty();
476  }
477  address oldAsset = redeemAsset;
478  redeemAsset = _address;
479  emit UpdateRedeemAsset(oldAsset, _address);
```

Recommendation

Enforce the documented invariants on-chain, matching **updateRedeemAsset**:



```
function updateMgtFeeTo(address _address) external onlyRole(MAINTAINER_ROLE) {
    if (_address == address(0)) revert InvalidAddress();
    if (totalMgtFeeUnclaimed != 0) revert InvalidState();
    if (pendingRedeemQueue.length() != 0 || redeemQueue.length() != 0) {
        revert QueuesNotEmpty();
    }
    mgtFeeTo = _address;
    emit UpdateMgtFeeTo(_address);
}
```

Status

✓ Fixed



[WP-L17] maxStalePeriod is not initialized and defaults to 0, silently disabling price staleness checks

Low

Issue Description

`getPrice()` guards staleness only when `maxStalePeriod > 0`. `initialize()` does not set `maxStalePeriod`, so it defaults to 0 and the staleness check is **entirely disabled at deployment time**. Unless MAINTAINER remembers to call `updateMaxStalePeriod(...)` as a separate post-deployment step, the contract silently accepts arbitrarily stale or compromised oracle prices for the lifetime of the deployment.

This is a silent failure mode: there is no revert and no event at deployment indicating that the staleness check is off. A reviewer reading only `getPrice` sees a `StalePriceData` revert and assumes staleness is enforced.

`getPrice` only enforces staleness if `maxStalePeriod > 0`:

```
625 function getPrice() public view returns (uint256 price) {
626     // If price oracle not set, return 1:1 ratio (1e18)
627     if (address(priceOracle) == address(0)) {
628         return 1e18;
629     }
630
631     (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound)
        = priceOracle.latestRoundData();
632
633     // Validate price is positive
634     if (answer <= 0) revert InvalidPrice(answer);
635
636     // Check for stale price data
637     if (maxStalePeriod > 0 && block.timestamp - updatedAt > maxStalePeriod) {
638         revert StalePriceData(updatedAt, block.timestamp, maxStalePeriod);
639     }
640 }
```

`initialize` never assigns `maxStalePeriod` (it stays at the default zero):

```
344 function initialize(
345     address _token,
346     address _redeemAsset,
347     address _treasury,
```



```
348 address _txFeeTo,
349 address _mgtFeeTo,
350 address admin,
351 address _assetRegistry,
352 address _priceOracle,
353 DepositRedeemLimiterCfg memory cfg
354 ) external initializer {
355     if (
356         admin == address(0) ||
357         _token == address(0) ||
358         _redeemAsset == address(0) ||
359         _treasury == address(0) ||
360         _txFeeTo == address(0) ||
361         _mgtFeeTo == address(0) ||
362         _assetRegistry == address(0)
363     ) revert InvalidAddress();
364
365     __AccessControlEnumerable_init();
366
367     token = IToken(_token);
368     redeemAsset = _redeemAsset;
369     treasury = _treasury;
370     txFeeTo = _txFeeTo;
371     mgtFeeTo = _mgtFeeTo;
372     assetRegistry = IAssetRegistry(_assetRegistry);
373     priceOracle = IPriceFeed(_priceOracle);
374
375     __DepositRedeemLimiter_init(cfg.depositMinimum, cfg.redeemMinimum,
376         cfg.firstDepositAmount);
377     _grantRole(DEFAULT_ADMIN_ROLE, admin);
378 }
```

Recommendation

Initialize `maxStalePeriod` in `initialize()` to a safe default, and remove the `> 0` guard so it cannot be unset. A value that bounds the T+2 BNY settlement cycle plus some slack (e.g. 3 days = 259200 seconds) is a reasonable starting point:



```
function initialize(...) external initializer {
  // ...
  maxStalePeriod = 3 days;
}

function getPrice() public view returns (uint256 price) {
  // ...
  if (block.timestamp - updatedAt > maxStalePeriod) {
    revert StalePriceData(updatedAt, block.timestamp, maxStalePeriod);
  }
  // ...
}
```

Additionally, consider a non-zero lower bound in `updateMaxStalePeriod` so MAINTAINER cannot set it back to 0.

Status

✓ Fixed



[WP-L18] processPendingRedeems() should likely revert when sender == mgtFeeTo && shareAmount > totalMgtFeeUnclaimed.

Low

Issue Description

This scenario is likely unintended and should not proceed further. It may be appropriate to cancel the pending redemption in such cases.

Otherwise, the corresponding decrement in functions like `cancelRedeem()` or `revertRedeemToPending()` (i.e., `totalMgtFeeUnclaimed += shareAmount;`) might fail to correctly restore the original state of `totalMgtFeeUnclaimed`.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L842-L934>

```
842  @@ 842,848 @@
849  function processPendingRedeems(uint256 _len) external onlyRole(OPERATOR_ROLE) {
850  uint256 processed;
851  uint256 currentPrice = getPrice();
852
853  while (processed < _len && !pendingRedeemQueue.empty()) {
854  if (!_processSinglePendingRedeem(currentPrice)) {
855  break;
856  }
857  unchecked {
858  ++processed;
859  }
860  }
861
862  if (processed == 0) revert NoPendingRedeemsReady();
863  }
864
865  @@ 865,869 @@
870  function _processSinglePendingRedeem(uint256 currentPrice) internal returns (bool
success) {
871  bytes memory data = pendingRedeemQueue.front();
872  (
873  address sender,
874  address receiver,
875  uint256 shareAmount, // Full share amount (no fee deduction)
876  uint256 requestTimestamp,
```



```
877     bytes32 pendingId
878 ) = _decodePendingRedeemData(data);
879
880 @@ 880,921 @@
922
923     redeemInfo[receiver] += shareAmount;
924     totalRedeemQueueShares += shareAmount;
925
926     // Decrement live mgt fee counter when fee tokens are priced in.
927     if (sender == mgtFeeTo && totalMgtFeeUnclaimed >= shareAmount) {
928         totalMgtFeeUnclaimed -= shareAmount;
929     }
930
931     emit ProcessPendingRedeem(sender, receiver, shareAmount, currentPrice,
932                               pendingId, finalId);
933
934     return true;
935 }
```

```
1069 @@ 1069,1073 @@
1074     function cancelRedeem(uint256 _len) public onlyRole(MAINTAINER_ROLE) {
1075         _len = _validateQueueProcessing(redeemQueue.length(), _len);
1076
1077         while (_len > 0) {
1078             bytes memory data = redeemQueue.popFront();
1079             (
1080                 address sender,
1081                 address receiver,
1082                 uint256 shareAmount, // Full share amount
1083                 ,
1084                 ,
1085                 ,
1086                 // redeemAssetAmt - not needed
1087                 // feeAssetAmt - not needed
1088                 // requestTimestamp - not needed
1089                 bytes32 id
1090             ) = _decodeRedeemData(data);
1091
1092             redeemInfo[receiver] -= shareAmount;
1093             totalRedeemQueueShares -= shareAmount;
1094             delete snapshotRatios[id];
1095
1096             // Restore the live mgt fee counter: pending->final had decremented it for
1097             fee-owned entries.
```



```
1097 // Cancelling from the final queue means those fee tokens are live again.
1098 if (sender == mgtFeeTo) {
1099     totalMgtFeeUnclaimed += shareAmount;
1100 }
1101
1102 @@ 1102,1114 @@
1115 }
1116 }

1151 @@ 1151,1158 @@
1159 function revertRedeemToPending(uint256 _len) external onlyRole(OPERATOR_ROLE) {
1160     _len = _validateQueueProcessing(redeemQueue.length(), _len);
1161
1162     // Pop from back of redeem queue and push to front of pending queue
1163     // This maintains order: redeemQueue [a, b] -> pendingQueue [a, b]
1164     for (uint256 i = 0; i < _len; ) {
1165         bytes memory data = redeemQueue.popBack();
1166         (
1167             address sender,
1168             address receiver,
1169             uint256 shareAmount, // Full share amount
1170             ,
1171             ,
1172             // redeemAssetAmt - discard (was calculated with wrong price)
1173             // feeAssetAmt - discard (was calculated with wrong price)
1174             uint256 requestTimestamp, // Keep original timestamp
1175             bytes32 oldId
1176         ) = _decodeRedeemData(data);
1177
1178         redeemInfo[receiver] -= shareAmount;
1179         totalRedeemQueueShares -= shareAmount;
1180
1181         // Restore the live mgt fee counter: pending->final had decremented it for
1182         // fee-owned entries.
1183         // Moving back to pending means those fee tokens are live again and will
1184         // be re-decremented
1185         // when they are re-processed through _processSinglePendingRedeem.
1186         if (sender == mgtFeeTo) {
1187             totalMgtFeeUnclaimed += shareAmount;
1188         }
1189     }
1190 }
```



Status

✓ Fixed



[WP-L19] If Express.circulatingSupply() is no longer being used, consider removing the circulatingSupply() function.

Low

Issue Description

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/5fe3eaf8b87d49e3ab7b7b6aa34ea5191cf14b7f/contracts/extension/Express.sol#L1460-L1470>

```
1460  /**
1461  * @notice Get circulating token supply
1462  * @dev Circulating = Total Supply - Shares committed to the final redeem queue -
1463  *      Live mgt fee tokens.
1464  *      Uses totalMgtFeeUnclaimed (live) so that fee redemptions don't move
1465  *      circulating. Moves only on
1466  *      user deposits, user redemptions reaching the final queue, and user-share
1467  *      burns.
1468  * @return supply Circulating token supply
1469  */
1470  function circulatingSupply() public view returns (uint256 supply) {
1471      uint256 totalSupply = IERC20(address(token)).totalSupply();
1472      supply = totalSupply - totalRedeemQueueShares - totalMgtFeeUnclaimed;
1473  }
```

Status

📌 Acknowledged



[WP-L20] The `depositInfo[_account]` returned by `Express.getDepositUserInfo()` is an aggregate of multiple `_asset` amounts, each potentially having different decimals and valuations.

Low

Issue Description

This structure is unintuitive and prone to misinterpretation.

The current implementation implicitly assumes that all `_asset` tokens share the same decimals and unit value.

However, the actual deposit logic correctly handles decimals and asset values: <https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L598-L600>

Depending on the intended use case for `getDepositUserInfo()`, I recommend either documenting the meaning of this return value clearly or refactoring the implementation to aggregate amounts by token or normalized decimal precision.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L464-L502>



```
464  /**
465  * @notice Queue a deposit request by depositing asset
466  * @dev Fee is charged at request time and net assets are forwarded to treasury
467  * @param _asset The asset token address (e.g., USDC, USDO)
468  * @param _amount Amount of asset deposited
469  * @param _receiver Address that receives minted token on processing
470  */
471  function requestDeposit(address _asset, uint256 _amount, address _receiver) external
  whenNotPausedDeposit {
472  @@ 472,492 @@
493  IERC20(_asset).safeTransferFrom(sender, treasury, netAmt);
494
495  depositInfo[_receiver] += netAmt;
496
497  bytes32 id = keccak256(abi.encode(_asset, sender, _receiver, netAmt, feeAmt,
  block.timestamp, _nonce++));
498  bytes memory data = abi.encode(_asset, sender, _receiver, netAmt, feeAmt, id);
499  depositQueue.pushBack(data);
500
501  emit AddToDepositQueue(_asset, sender, _receiver, netAmt, feeAmt, id);
502  }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L751-L758>

```
751  /**
752  * @notice Get user's total queued deposit amount
753  * @param _account User address
754  * @return amount Total asset amount pending mint
755  */
756  function getDepositUserInfo(address _account) external view returns (uint256
  amount) {
757  return depositInfo[_account];
758  }
```

Status

✓ Fixed



[WP-L21] PriceOracle Deviation Formula Uses Symmetric Average Instead of Base-Relative

Low

Issue Description

`_calculateDeviation()` uses `(base + new) / 2` as the denominator instead of `base`. This is a symmetric difference metric, not a standard basis-point deviation from the base price. As a result, configured thresholds are looser on upward moves and tighter on downward moves than operators would expect. For example, a configured 1000 bps (10%) limit permits approximately +10.53% upward but only -9.52% downward.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/PriceOracle.sol#L262-L269>

```
262 function _calculateDeviation(uint256 _basePrice, uint256 _newPrice) private pure
    returns (uint256 deviation) {
263     uint256 numerator = _basePrice > _newPrice ? _basePrice - _newPrice : _newPrice
        - _basePrice;
264     uint256 denominator = (_basePrice + _newPrice) / 2;
265
266     @@ 266,267 @@
268     deviation = (numerator * DEVIATION_FACTOR) / denominator;
269 }
```

PoC

```
// With relativeMaxDeviation = 1000 (10%)
// base = 100e8, new = 111e8 (11% up from base)

// Expected: deviation = |100 - 111| / 100 * 10000 = 1100 bps > 1000 => REJECT
// Actual:    deviation = |100 - 111| / ((100 + 111) / 2) * 10000
//            = 11 / 105 * 10000 = 1047 bps > 1000 => REJECT (correct)

// base = 100e8, new = 110.5e8 (10.5% up from base)
// Expected: deviation = 10.5 / 100 * 10000 = 1050 bps > 1000 => REJECT
// Actual:    deviation = 10.5 / 105.25 * 10000 = 997 bps < 1000 => ACCEPT (wrong!)
```

Recommendation

Use `_basePrice` as the denominator for standard relative deviation measurement.



```
function _calculateDeviation(uint256 _basePrice, uint256 _newPrice) private pure
returns (uint256 deviation) {
    uint256 numerator = _basePrice > _newPrice ? _basePrice - _newPrice : _newPrice
    - _basePrice;
    -      uint256 denominator = (_basePrice + _newPrice) / 2;
    +      uint256 denominator = _basePrice;

    if (denominator == 0) return 0;

    deviation = (numerator * DEVIATION_FACTOR) / denominator;
}
```

Status

① Acknowledged



[WP-L22] PriceOracle.confirmPrice() fails to specify the expected observedAt and proposedAt.

Low

Issue Description

An operator could front-run the transaction or repeatedly call `proposePrice()` with the same `value`.

This allows submitting an `observedAt` value outside of the confirmer's expectations and bypassing the `block.timestamp <= proposedAt + PENDING_PRICE_TTL` constraint.

Consider adding `expectedObservedAt` to `confirmPrice()` as well.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/PriceOracle.sol#L250-L277>

```
250  function confirmPrice(int256 _expectedPrice) external onlyRole(CONFIRMER_ROLE) {
251  if (!pendingPriceValue.exists) revert PendingPriceNotSet();
252  if (pendingPriceValue.value != _expectedPrice) revert PriceMismatch-
    (_expectedPrice, pendingPriceValue.value);
253  uint256 expiresAt = pendingPriceValue.proposedAt + PENDING_PRICE_TTL;
254  if (block.timestamp > expiresAt) {
255  revert PendingPriceExpired(pendingPriceValue.proposedAt, expiresAt,
    block.timestamp);
256  }
257
258  int256 newPrice = pendingPriceValue.value;
259  int256 oldAnswer = latestAnswer();
260
261  emit UpdatePrice(oldAnswer, newPrice);
262
263  unchecked {
264  ++latestRoundValue;
265  }
266
267  RoundData storage round = roundData[latestRoundValue];
268  round.roundId = latestRoundValue;
269  round.answer = newPrice;
270  round.startedAt = pendingPriceValue.observedAt;
271  round.updatedAt = pendingPriceValue.observedAt;
272  round.answeredInRound = latestRoundValue;
273
```



```
274     delete pendingPriceValue;
275
276     emit RoundUpdated(latestRoundValue);
277 }

224     function proposePrice(int256 _price, uint256 _priceTimestamp) external onlyRole-
(OPERATOR_ROLE) {
225         if (_price <= 0) revert InvalidPrice();
226         if (_priceTimestamp == 0 || _priceTimestamp >= block.timestamp) revert
InvalidTimestamp();
227
228         int256 latestPrice = latestAnswer();
229         uint256 priceDeviation = _calculateDeviation(uint256(latestPrice), uint256-
(_price));
230         if (priceDeviation > relativeMaxDeviationValue) {
231             revert RelativeDeviationTooLarge(priceDeviation, relativeMaxDeviationValue);
232         }
233
234         uint256 referenceDeviation = _calculateDeviation(referencePriceValue,
uint256(_price));
235         if (referenceDeviation > absoluteMaxDeviationValue) {
236             revert AbsoluteDeviationTooLarge(referenceDeviation,
absoluteMaxDeviationValue);
237         }
238
239         pendingPriceValue = PendingPrice({
240             value: _price,
241             proposedAt: block.timestamp,
242             proposer: msg.sender,
243             exists: true,
244             observedAt: _priceTimestamp
245         });
246
247         emit PriceProposed(latestPrice, _price, _priceTimestamp, msg.sender);
248     }
```

Status

📄 Acknowledged



[WP-L23] The updateRedeemAsset and initialize functions fail to verify whether _address is supported by assetRegistry.

Low

Issue Description

This can lead to configuration inconsistencies, potentially bricking the redeem process.

Express.sol

```
425 function updateRedeemAsset(address _address) external onlyRole(MAINTAINER_ROLE) {
426     if (_address == address(0)) revert InvalidAddress();
427     if (depositQueue.length() != 0 || pendingRedeemQueue.length() != 0 ||
        redeemQueue.length() != 0) {
428         revert QueuesNotEmpty();
429     }
430     address oldAsset = redeemAsset;
431     redeemAsset = _address;
432     emit UpdateRedeemAsset(oldAsset, _address);
433 }
```

Express.sol

```
307 function initialize(
308     address _token,
309     address _redeemAsset,
310     address _treasury,
311     address _txFeeTo,
312     address _mgtFeeTo,
313     address admin,
314     address _assetRegistry,
315     address _priceOracle,
316     DepositRedeemLimiterCfg memory cfg
317 ) external initializer {
318     if (
319         admin == address(0) ||
320         _token == address(0) ||
321         _redeemAsset == address(0) ||
322         _treasury == address(0) ||
323         _txFeeTo == address(0) ||
324         _mgtFeeTo == address(0) ||
325         _assetRegistry == address(0)
```



```
326 ) revert InvalidAddress();
327
328 __AccessControlEnumerable_init();
329
330 token = IToken(_token);
331 redeemAsset = _redeemAsset;
332 treasury = _treasury;
333 txFeeTo = _txFeeTo;
334 mgtFeeTo = _mgtFeeTo;
335 assetRegistry = IAssetRegistry(_assetRegistry);
336 priceOracle = IPriceFeed(_priceOracle);
337
338 __DepositRedeemLimiter_init(cfg.depositMinimum, cfg.redeemMinimum,
    cfg.firstDepositAmount);
339
340 _grantRole(DEFAULT_ADMIN_ROLE, admin);
341 }
```

AssetRegistry.sol

```
114 function isAssetSupported(address asset) external view returns (bool supported) {
115     return _assetConfigs[asset].isSupported;
116 }
```

Redeem ~~if~~ revert

Express.sol

```
1477 function _redeemAssetAmount(
1478     uint256 _shareAmount,
1479     uint256 _ratio,
1480     uint256 _price
1481 ) internal view returns (uint256 redeemAssetAmt) {
1482     uint256 backedShareAmount = Math.mulDiv(_shareAmount, _ratio, 1e18);
1483     redeemAssetAmt = _trimAsset(
1484         Math.mulDiv(convertToUnderlying(redeemAsset, backedShareAmount), _price,
            1e18),
1485         redeemAsset
1486     );
1487 }
```

Express.sol



```
1409     function convertToUnderlying(address _token, uint256 _amount) public view returns
        (uint256 amount) {
1410         return assetRegistry.convertToUnderlying(_token, _amount);
1411     }
```

AssetRegistry.sol

```
134
135     function convertToUnderlying(address _asset, uint256 _amount) external view returns
        (uint256 amount) {
136         AssetConfig memory config = _assetConfigs[_asset];
137         if (!config.isSupported) revert AssetRegistryAssetNotSupported(_asset);
138
139         uint8 assetDecimals = IERC20Metadata(_asset).decimals();
140
141         // Scale down from 18 decimals to asset decimals with proper rounding
142         uint256 divisor = 10 ** (DECIMALS - assetDecimals);
143         uint256 amount1 = _amount.mulDiv(1, divisor, Math.Rounding.Floor);
144
145         // If asset has price feed, convert from USD value
146         if (config.priceFeed != address(0)) {
147             (uint256 rate, uint8 feedDecimals) = _getFreshPrice(_asset, config.priceFeed);
148             amount = amount1.mulDiv(10 ** feedDecimals, rate);
149         } else {
150             amount = amount1;
151         }
152     }
```

Status

① Acknowledged



[WP-L24] DoubleQueueModified derives from an older version of the OpenZeppelin DoubleEndedQueue and inherits some of its original issues.

Low

Issue Description

For instance:

The `empty()` function is implemented as `return deque._end <= deque._begin;`. Theoretically, if `pushBack()` (L47-L56) increments `deque._end` beyond `type(int128).max`, `empty()` will incorrectly return `true`. This creates an inconsistency with the `length()` implementation at L154, which uses `unchecked { return uint256(int256(deque._end) - int256(deque._begin)); }`.

In this scenario, `empty()` will continue to return `true` even after subsequent calls to `pushBack()`.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/60266fd8b77decc5391b6bb80a3e1aa51852e240/contracts/extension/DoubleQueueModified.sol/#L147-L163>

```
147  /**
148   * @dev Returns the number of items in the queue.
149   */
150   function length(BytesDeque storage deque) internal view returns (uint256) {
151     // The interface preserves the invariant that begin <= end so we assume this
152     // will not overflow.
153     // We also assume there are at most int256.max items in the queue.
154     unchecked {
155       return uint256(int256(deque._end) - int256(deque._begin));
156     }
157   }
158   /**
159   * @dev Returns true if the queue is empty.
160   */
161   function empty(BytesDeque storage deque) internal view returns (bool) {
162     return deque._end <= deque._begin;
163   }
```



<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/60266fd8b77decc5391b6bb80a3e1aa51852e240/contracts/extension/DoubleQueueModified.sol#L47-L56>

```
47  /**
48  * @dev Inserts an item at the end of the queue.
49  */
50  function pushBack(BytesDeque storage deque, bytes memory value) internal {
51      int128 backIndex = deque._end;
52      deque._data[backIndex] = value;
53      unchecked {
54          deque._end = backIndex + 1;
55      }
56  }
```

Since OpenZeppelin 4.6.0, the **DoubleEndedQueue** library has undergone several refinements:

- <https://github.com/OpenZeppelin/openzeppelin-contracts/pull/4150>
- <https://github.com/OpenZeppelin/openzeppelin-contracts/pull/4872>
- <https://github.com/OpenZeppelin/openzeppelin-contracts/pull/6020>

Referencing the latest OpenZeppelin 5.6.1 implementation of **DoubleEndedQueue** will provide cleaner, more consistent logic and better gas efficiency.

Status

✓ Fixed



[WP-L25] PriceOracle uses unique storage slots for each round to preserve historical data, yet it lacks the `AggregatorV3Interface.getRoundData(uint80 _roundId)` interface.

Low

Issue Description

- If historical round data support is required: Consider implementing the Chainlink `AggregatorV3Interface.getRoundData(uint80 _roundId)` interface.
- If historical round data support is unnecessary: Consider using a single storage slot for round data to reduce unnecessary gas consumption.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/67d457005a96f66960422e44009383bd2fa2b834/contracts/extension/PriceOracle.sol#L8-L286>

```
8  /**
9   * @title PriceOracle
10  * @notice Upgradeable oracle contract for managing staged price updates with deviation
    controls
11  * @dev Uses UUPS upgradeable pattern for future improvements
12  *
13  * Key Features:
14  * - Two configurable price deviation limits for safety
15  * - Two-step price updates (proposal + confirmation)
16  * - Round-based price updates (Chainlink-compatible interface)
17  * - Role-based access control
18  * - Upgradeable via UUPS pattern
19  */
20  contract PriceOracle is Initializable, AccessControlEnumerableUpgradeable,
    UUPSUpgradeable {
21  @@ 21,47 @@
48  mapping(uint80 => RoundData) private roundData;
49  PendingPrice private pendingPriceValue;
50
51  @@ 51,74 @@
75
76  @@ 76,84 @@
85  function initialize(
86  uint8 _decimals,
```



```
87  uint256 _relativeMaxDeviation,
88  uint256 _absoluteMaxDeviation,
89  uint256 _initPrice,
90  uint256 _referencePrice,
91  address _admin
92  ) external initializer {
93  @@ 93,104 @@
105
106  latestRoundValue = 1;
107  RoundData storage round = roundData[latestRoundValue];
108  round.roundId = latestRoundValue;
109  round.answer = _initPrice;
110  round.startedAt = block.timestamp;
111  round.updatedAt = block.timestamp;
112  round.answeredInRound = latestRoundValue;
113
114  _grantRole(DEFAULT_ADMIN_ROLE, _admin);
115  }
116
117  /*//////////////////////////////////////
118  VIEW FUNCTIONS
119  //////////////////////////////////////*/
120
121  function decimals() public view returns (uint8) {
122  return decimalsValue;
123  }
124
125  function latestRound() public view returns (uint80) {
126  return latestRoundValue;
127  }
128
129  function latestRoundData() public view returns (uint80, uint256, uint256, uint256,
uint80) {
130  RoundData storage round = roundData[latestRoundValue];
131  return (round.roundId, round.answer, round.startedAt, round.updatedAt,
round.answeredInRound);
132  }
133
134  function latestAnswer() public view returns (uint256) {
135  return roundData[latestRoundValue].answer;
136  }
137
138  @@ 138,229 @@
230
```



```
231     function confirmPrice(uint256 _expectedPrice) external onlyRole(CONFIRMER_ROLE) {
232     @@ 232,240 @@
241
242         emit UpdatePrice(oldAnswer, newPrice);
243
244         unchecked {
245             ++latestRoundValue;
246         }
247
248         RoundData storage round = roundData[latestRoundValue];
249         round.roundId = latestRoundValue;
250         round.answer = newPrice;
251         round.startedAt = block.timestamp;
252         round.updatedAt = block.timestamp;
253         round.answeredInRound = latestRoundValue;
254
255         delete pendingPriceValue;
256
257         emit RoundUpdated(latestRoundValue);
258     }
259
260     @@ 260,285 @@
286 }
```

Status

✓ Fixed



[WP-I26] Operational hazard: `updateOffchainShares` accepts arbitrary values while queues are non-empty — recommend on-chain queue-empty guard

Informational

Issue Description

`updateOffchainShares` is documented as a rare admin override for share splits and reconciliation. The function ships with no on-chain guards — `MAINTAINER_ROLE` can call it at any time, including while `depositQueue`, `pendingRedeemQueue`, or `redeemQueue` is non-empty.

This is **not** an adversarial / privilege-escalation finding. `MAINTAINER_ROLE` already controls `processDepositQueue` directly, and the role is trusted to sequence operations atomically (drain queues first, then call `updateOffchainShares`, then accept new requests). A malicious MAINTAINER has many easier paths to abuse the protocol; this finding is not about MAINTAINER trust.

The concern is **operational error**. When the queues are non-empty, `offchainShares` is mid-state — it has been incremented by `processDepositQueue` calls and decremented by `requestRedeem` calls, and there are queue entries with `shareAmount` already deducted from `offchainShares`. Picking the "correct" value for a manual override in that state is non-trivial:

- For a corporate-action share split, the operator must decide whether the new value reflects the post-split count of *currently held* shares (excluding queued redeems) or *all owed* shares (including queued redeems whose `shareAmount` has been pre-deducted).
- For a reconciliation override, any difference between the override value and the value implied by in-flight queue entries propagates into `processDepositQueue`'s `currentRatio` snapshot at the next call, mispricing the queued batch.
- The function emits `UpdateOffchainShares(caller, newValue, previousValue)`, but neither the event nor the function signature surfaces the queue state at the time of the call, so an off-chain reviewer cannot easily catch the misuse after the fact.

The natural operational pattern is: drain all queues → call `updateOffchainShares` → accept new requests. Forcing this pattern on-chain removes the foot-gun without restricting any legitimate use case.



Vulnerable Code

```
1558 function updateOffchainShares(uint256 _newValue) external onlyRole(MAINTAINER_ROLE)
1559 {
1560     uint256 previous = offchainShares;
1561     offchainShares = _newValue;
1562     emit UpdateOffchainShares(_msgSender(), _newValue, previous);
1563 }
```

The same `_requireQueuesEmpty()` pattern is already applied to other state-altering setters whose value is meaningful only against an empty-queue baseline (`updateRedeemAsset`, `updatePriceOracle`, `updateMaxStalePeriod`, `updateDepositMaxDeviationBps`, etc.). Adding it here makes the operational invariant explicit and self-enforcing.

Recommendation

Apply the same queue-empty guard:

```
function updateOffchainShares(uint256 _newValue) external onlyRole(MAINTAINER_ROLE) {
    _requireQueuesEmpty();
    uint256 previous = offchainShares;
    offchainShares = _newValue;
    emit UpdateOffchainShares(_msgSender(), _newValue, previous);
}
```

The intended operational sequence (drain → update → reopen) becomes trivially supported, and the misuse path (fat-finger override during in-flight settlement) is rejected at the EVM level instead of relying on runbook discipline.

Status

✓ Fixed



[WP-I27] Operational hazard: Token.setKycManager has no queue-empty / cross-contract consistency guard — recommend mirroring Express's rotation discipline on-chain

Informational

Issue Description

Express's `setKycManager` requires queues empty before allowing rotation (-`Express.sol:420-426`). Token's equivalent does not — `DEFAULT_ADMIN_ROLE` can rotate `Token.kycManager` at any time, including while Express has non-empty queues mid-settlement.

This is **not** an adversarial finding. `DEFAULT_ADMIN_ROLE` controls both setters and is trusted to sequence rotation correctly: drain Express queues → rotate Token's manager → rotate Express's manager → reopen. A malicious admin has many easier paths; this is purely about the rotation runbook being enforced off-chain rather than on-chain.

The concern is operational error. Token's `_update` calls `IKycManager(kycManager).isKycd(...)` for every mint, burn, and transfer. Three settlement paths transit through that gate:

- `processDepositQueue` → `token.mint(receiver, mintedAmount)` checks `isKycd(receiver)`.
- `processRedeemQueue` → `token.burn(address(this), tokenAmount)` checks `isKycd(this)` (Express itself).
- `requestRedeem` → `token.safeTransferFrom(from, address(this), _tokenAmount)` checks both sides.

If admin rotates Token's manager mid-flight without first draining Express's queues, the next call into any of these paths reverts. Examples of accidental misuse:

- Rotating Token to a fresh manager that hasn't KYC'd Express's address yet — every settlement reverts on `NotKycd(express)`.
- Rotating Token while Express stays on the old manager — Express's `_validateKyc` passes against the old manager, but `Token._update` rejects against the new one. The mismatch wedges the front of whichever queue is being processed.
- Toggling Token to permissionless and back during a deploy/test sequence, leaving in-flight entries gated inconsistently.



Express has the queue-empty discipline encoded on-chain. Adding the same to Token closes the cross-contract symmetry and removes the off-chain coordination burden.

There is also a structural question worth raising separately: Token's KYC gate is independent of Express's, but Token's **MINTER_ROLE** and **BURNER_ROLE** are granted only to Express, so the mint/-burn paths are already gated by Express's KYC enforcement. The Token-level check only adds value for direct user-to-user HYBOND transfers — which could be served by reading Express's manager rather than holding an independent reference. Consolidating to a single source of truth would also remove the desync vector entirely.

Vulnerable Code

Token's setter is unguarded:

```
169  function setKycManager(address newManager) external onlyRole(DEFAULT_ADMIN_ROLE)
    {
170      address old = kycManager;
171      kycManager = newManager;
172      emit KycManagerUpdated(old, newManager);
173  }
```

Express's setter already enforces the discipline:

```
420  function setKycManager(address newManager) external onlyRole(DEFAULT_ADMIN_ROLE)
    {
421      if (newManager == address(0)) revert InvalidAddress();
422      _requireQueuesEmpty();
423      address old = kycManager;
424      kycManager = newManager;
425      emit KycManagerUpdated(old, newManager);
426  }
```

The Token gate that Express settlement transits through:



```
257 }
258
259 // KYC gate (permissionless when kycManager == address(0))
260 address mgr = kycManager;
261 if (mgr != address(0)) {
262     if (from != address(0) && !IKycManager(mgr).isKycd(from)) revert
    NotKycd(from);
263     if (to != address(0) && !IKycManager(mgr).isKycd(to)) revert NotKycd(to);
264 }
265
```

Recommendation

Mirror Express's discipline on Token's side. Two viable directions:

Option A — match Express's queue-empty guard. Hold a reference to the linked Express and require its queues empty before allowing rotation:

```
interface IExpressQueueLengths {
    function getDepositQueueLength() external view returns (uint256);
    function getPendingRedeemQueueLength() external view returns (uint256);
    function getRedeemQueueLength() external view returns (uint256);
}

function setKycManager(address newManager) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (express != address(0)) {
        if (
            IExpressQueueLengths(express).getDepositQueueLength() != 0 ||
            IExpressQueueLengths(express).getPendingRedeemQueueLength() != 0 ||
            IExpressQueueLengths(express).getRedeemQueueLength() != 0
        ) revert QueuesNotEmpty();
    }
    address old = kycManager;
    kycManager = newManager;
    emit KycManagerUpdated(old, newManager);
}
```

Option B — drop Token's independent KYC source. Have Token read Express's `kycManager()` (or a single shared singleton) instead of holding its own. This eliminates the desync surface entirely; rotation becomes a single-contract operation gated by Express's existing queue-empty rule.

Option B is structurally cleaner; option A is a smaller diff. Either approach turns the off-chain rotation runbook into an on-chain invariant.



Status

 Acknowledged



[WP-I28] Both `depositMaxDeviationBps` and `redeemMaxDeviationBps` default to zero on initialize, and the setters require queues empty — deployment-day self-DoS trap

Informational

Issue Description

The round-2 design spec

([docs/superpowers/specs/2026-04-29-redeem-distribution-deviation-design.md](#)) says fresh deployments should initialize both bps fields to `100` (1%) inside `initialize`. The implementation does not do this. Solidity's default zero remains in place, and `_checkDeviation(actual, expected, 0)` reverts on any non-exact match (`expected != actual`).

Because oracle-derived `expectedTotal` (sum of per-entry `_redeemAssetAmount` calls, each with `mulDiv` floor + `_trimAsset` truncation) effectively never matches the off-chain-computed `_totalAsset` to the wei, the first oracle-backed batch reverts `OracleDeviationExceeded`.

The corrective setters (`updateDepositMaxDeviationBps`, `updateRedeemMaxDeviationBps`) both call `_requireQueuesEmpty()`. So once the first user submits a deposit or redeem, the bps fields cannot be updated until the queues are drained — typically requiring `cancelDeposit` / `cancelPendingRedeem` (MAINTAINER_ROLE), which refund the user via escrow paths. After draining, set bps, then users re-submit.

Sequence on mainnet day one:

1. Deploy Express. `depositMaxDeviationBps = 0`, `redeemMaxDeviationBps = 0`.
2. First user submits `requestDeposit(...)`. `depositQueue.length() == 1`.
3. MAINTAINER calls `processDepositQueue(1, _newShares)`. `_checkDeviation(_newShares, oracleShares, 0)` reverts on any rounding drift.
4. MAINTAINER calls `updateDepositMaxDeviationBps(100)`. Reverts on `_requireQueuesEmpty()`.
5. Recovery: `cancelDeposit(1)` → user gets refund via `depositEscrowBalance`, queue empty, `updateDepositMaxDeviationBps(100)` succeeds, user re-submits. Process repeats on the redeem side.



The deploy script `deploy/04_deploy_express.ts` post-init configures `convertRedeemRequestsDelay` and `timeBuffer` but does not set the bps fields, so the protocol comes up in strict-equality mode by default.

`initialize` does not set the bps fields:

```
348 function initialize(  
349     address _token,  
350     address _redeemAsset,  
351     address _treasury,  
352     address _txFeeTo,  
353     address _mgtFeeTo,  
354     address admin,  
355     address _assetRegistry,  
356     address _priceOracle,  
357     uint256 _maxStalePeriod,  
358     DepositRedeemLimiterCfg memory cfg,  
359     address _kycManager  
360 ) external initializer {  
361     if (  
362         admin == address(0) ||  
363         _token == address(0) ||  
364         _redeemAsset == address(0) ||  
365         _treasury == address(0) ||  
366         _txFeeTo == address(0) ||  
367         _mgtFeeTo == address(0) ||  
368         _assetRegistry == address(0) ||  
369         _priceOracle == address(0) ||  
370         _kycManager == address(0)  
371     ) revert InvalidAddress();  
372  
373     __AccessControlEnumerable_init();  
374  
375     token = IToken(_token);  
376     redeemAsset = _redeemAsset;  
377     treasury = _treasury;  
378     txFeeTo = _txFeeTo;  
379     mgtFeeTo = _mgtFeeTo;  
380     assetRegistry = IAssetRegistry(_assetRegistry);  
381     priceOracle = IPriceFeed(_priceOracle);  
382     maxStalePeriod = _maxStalePeriod;  
383     kycManager = _kycManager;  
384
```



```
385 __DepositRedeemLimiter_init(cfg.depositMinimum, cfg.redeemMinimum,  
386   cfg.firstDepositAmount);  
387 __grantRole(DEFAULT_ADMIN_ROLE, admin);  
388 }
```

The setters with `_requireQueuesEmpty` make recovery impossible without first cancelling outstanding requests:

```
549  */  
550  function updateDepositMaxDeviationBps(uint256 _bps) external onlyRole(-  
551    MAINTAINER_ROLE) {  
552    if (_bps > BPS_BASE) revert InvalidInput(_bps);  
553    __requireQueuesEmpty();  
554    depositMaxDeviationBps = _bps;  
555    emit UpdateDepositMaxDeviationBps(_bps);  
556  }  
557  /**  
558  * @notice Update the symmetric oracle deviation tolerance for processPendingRedeems  
559  * @param _bps Deviation tolerance in basis points (0 = strict, max = BPS_BASE)  
560  */  
561  function updateRedeemMaxDeviationBps(uint256 _bps) external onlyRole(-  
562    MAINTAINER_ROLE) {  
563    if (_bps > BPS_BASE) revert InvalidInput(_bps);  
564    __requireQueuesEmpty();  
565    redeemMaxDeviationBps = _bps;  
566    emit UpdateRedeemMaxDeviationBps(_bps);  
567  }
```

Recommendation

Initialize both fields in `initialize` to the spec's intended default (1% = 100 bps), as the design doc explicitly required:

```
function initialize(  
  ...  
) external initializer {  
  // ... existing checks and assignments ...  
  depositMaxDeviationBps = 100;  
  redeemMaxDeviationBps = 100;  
  // ... rest unchanged ...  
}
```



Or, if the policy choice should be deploy-time configurable, add the two values as initializer parameters and let the deploy script pass `100` explicitly (mirrors how `_maxStalePeriod` is handled). Either approach removes the silent strict-equality default that locks the protocol on first batch.

Status

✓ Fixed



[WP-I29] Cancelled first deposit leaves firstDeposit permanently set

Informational

Issue Description

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L471-L502>



```
471 function requestDeposit(address _asset, uint256 _amount, address _receiver) external
whenNotPausedDeposit {
472     address sender = _msgSender();
473     if (_amount == 0) revert InvalidAmount();
474
475     _validateKyc(sender, _receiver);
476
477     uint256 equivalentAmount = convertFromUnderlying(_asset, _amount);
478     if (!firstDeposit[sender]) {
479         if (equivalentAmount < firstDepositAmount) {
480             revert FirstDepositLessThanRequired(equivalentAmount, firstDepositAmount);
481         }
482         firstDeposit[sender] = true;
483     } else if (equivalentAmount < depositMinimum) {
484         revert DepositLessThanMinimum(equivalentAmount, depositMinimum);
485     }
486
487     uint256 feeAmt = txsFee(_amount, TxType.DEPOSIT);
488     uint256 netAmt = _amount - feeAmt;
489
490     if (feeAmt > 0) {
491         IERC20(_asset).safeTransferFrom(sender, txFeeTo, feeAmt);
492     }
493     IERC20(_asset).safeTransferFrom(sender, treasury, netAmt);
494
495     depositInfo[_receiver] += netAmt;
496
497     bytes32 id = keccak256(abi.encode(_asset, sender, _receiver, netAmt, feeAmt,
block.timestamp, _nonce++));
498     bytes memory data = abi.encode(_asset, sender, _receiver, netAmt, feeAmt, id);
499     depositQueue.pushBack(data);
500
501     emit AddToDepositQueue(_asset, sender, _receiver, netAmt, feeAmt, id);
502 }
```

Express.sol

```
615 function cancelDeposit(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
616     _len = _validateQueueProcessing(depositQueue.length(), _len);
617
618     while (_len > 0) {
619         bytes memory data = depositQueue.front();
620         (
621             address asset,
```



```
622     address sender,
623     address receiver,
624     uint256 netAssets,
625     uint256 feeAmt,
626     bytes32 prevId
627 ) = _decodeDepositData(data);
628
629     uint256 refundAmt = netAssets + feeAmt;
630     uint256 availableBalance = getTokenBalance(asset);
631     if (availableBalance < refundAmt) {
632         revert InsufficientLiquidity(refundAmt, availableBalance);
633     }
634
635     depositQueue.popFront();
636     depositInfo[receiver] -= netAssets;
637
638     unchecked {
639         --_len;
640     }
641
642     IERC20(asset).safeTransfer(sender, refundAmt);
643
644     emit CancelProcessDeposit(asset, sender, receiver, netAssets, feeAmt,
645                               prevId);
646 }
```

Description

`requestDeposit()` marks `firstDeposit[sender] = true` before the deposit is ever minted. If the queued deposit is later cancelled by a maintainer, the refund logic does not clear that flag. The account is therefore treated as having completed its first deposit even though no successful mint occurred.

Status

📄 Acknowledged



[WP-I30] AssetRegistry Initializer Grants MAINTAINER_ROLE to msg.sender

Informational

Issue Description

`AssetRegistry.initialize()` grants `MAINTAINER_ROLE` to both the `admin` parameter and `msg.sender`. If the deployer differs from admin (e.g., a deployment script, factory contract, or hardhat-deploy proxy), the deployer retains permanent maintainer powers over asset configurations and price feeds. A maintainer can add malicious price feeds via `setAssetConfig()` or remove supported assets via `removeAsset()`, directly impacting deposit/redeem pricing through Express.

```
// Deployer != admin
assetRegistry.initialize(multisig); // msg.sender = deployerEOA

// deployerEOA now has MAINTAINER_ROLE
assetRegistry.setAssetConfig(AssetConfig({
  asset: usdc,
  isSupported: true,
  priceFeed: maliciousFeed, // returns manipulated price
  maxStalePeriod: 86400
}));
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/AssetRegistry.sol#L43-L51>

```
43  function initialize(address admin) external initializer {
44  @@ 44,46 @@
47  _grantRole(DEFAULT_ADMIN_ROLE, admin);
48  _grantRole(MAINTAINER_ROLE, admin);
49  _grantRole(MAINTAINER_ROLE, msg.sender);
50  @@ 50,51 @@
```

Recommendation

Remove the `msg.sender` grant, or require `msg.sender == admin` as a precondition. If `msg.sender` is intentionally granted the role for deployment convenience, revoke it in a post-deployment script.



```
function initialize(address admin) external initializer {
  __AccessControl_init();
  __UUPSUpgradeable_init();

  _grantRole(DEFAULT_ADMIN_ROLE, admin);
  _grantRole(MAINTAINER_ROLE, admin);
  -   _grantRole(MAINTAINER_ROLE, msg.sender);
  _grantRole(UPGRADE_ROLE, admin);
}
```

Status

✓ Fixed



[WP-I31] Operator Can Inflate Management Fees via updateEpoch Override

Informational

Issue Description

`updateEpoch()` accepts an operator-supplied `_circulatingSupply` override and only validates `<= totalSupply()`. It does not enforce `<= circulatingSupply()`, so the operator can ignore tokens already excluded by the protocol's own accounting (especially `totalRedeemQueueShares` and tokens sitting at `mgtFeeTo`) and accrue fees on a larger base than intended.

PoC

1. Protocol has `totalSupply = 1_000_000e18`, `totalRedeemQueueShares = 200_000e18`, `mgtFeeTo balance = 50_000e18`.
2. True `circulatingSupply() = 750_000e18`.
3. Operator calls `updateEpoch(1_000_000e18)` passing `totalSupply` as override.
4. Daily fee is calculated on `1_000_000e18` instead of `750_000e18`, a 33% inflation.
5. Over many epochs, the operator mints excess fee claims via `claimMgtFee()`.

```
// Honest call
express.updateEpoch(0); // uses circulatingSupply() = 750_000e18

// Dishonest call
express.updateEpoch(1_000_000e18); // passes validation (== totalSupply)
// dailyFee is 33% higher than it should be
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/d2e32fa40327fb427c7bbb9980fa1e957db46ab8/contracts/extension/Express.sol#L1358-L1382>



```
1358 function updateEpoch(uint256 _circulatingSupply) external onlyRole(OPERATOR_ROLE)
1359 {
1360     @@ 1359,1368 @@
1369     if (_circulatingSupply > 0) {
1370         uint256 totalSupply = IERC20(address(token)).totalSupply();
1371         if (_circulatingSupply > totalSupply) revert InvalidInput(_circulatingSupply);
1372         circulating = _circulatingSupply;
1373     } else {
1374         circulating = circulatingSupply();
1375     }
1376     @@ 1375,1382 @@
```

Recommendation

Cap the override at `circulatingSupply()` rather than `totalSupply()`, or remove the override entirely. If an override is needed for off-chain data, restrict it to a tightly bounded deviation from on-chain `circulatingSupply()`.

```
if (_circulatingSupply > 0) {
-     uint256 totalSupply = IERC20(address(token)).totalSupply();
-     if (_circulatingSupply > totalSupply) revert InvalidInput(_circulatingSupply);
+     uint256 onChainCirculating = circulatingSupply();
+     if (_circulatingSupply > onChainCirculating) revert
InvalidInput(_circulatingSupply);
    circulating = _circulatingSupply;
}
```

Status

✓ Fixed



[WP-I32] A single price for the buy and sell price of the BNY ETF may not be enough.

Informational

Issue Description

If the buy and sell prices for the BNY ETF differ due to spread volatility, you may need to implement separate **PriceOracle** instances for each.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/doc/plan/management-fee.md#L1-L23>

```
1  # Overview
2
3  Hybond is a token issued via `Token.sol`, used to tokenize real-world assets: a BNY
  ETF Fund.
4
5  ## Token Mint Workflow (Express.sol:requestDeposit)
6
7  1. User calls `requestDeposit` to deposit USDC to the contract at T+0.
8  2. Fund operator transfers the USDC to buy BNY ETF Fund off-chain at T+0 EOD.
9  3. At T+2, BNY emails us the share price and share amount of the BNY ETF Fund.
10 4. Fund operator updates the share price in `PriceOracle.sol` via `proposePrice` and
   `confirmPrice`.
11 5. Fund operator calls `processDepositQueue` (MAINTAINER_ROLE) to mint Hybond tokens
   to users.
12 6. User receives Hybond tokens in their wallet.
13
14 ## Token Redeem Workflow (Express.sol:requestRedeem)
15
16 1. User calls `requestRedeem` to redeem Hybond for USDC at T+0.
17 2. Fund operator calls `sharesPerToken` to calculate the off-chain shares to sell.
18 3. Fund operator sells BNY ETF Fund shares off-chain at T+0 EOD.
19 4. At T+2, BNY emails us the sell price and USDC amount we will receive.
20 5. Fund operator updates the price in `PriceOracle.sol` via `proposePrice` and
   `confirmPrice`.
21 6. Fund operator calls `processPendingRedeems` (OPERATOR_ROLE) to lock in USDC
   amounts using the current price and `sharesPerToken` ratio. Requests move from
   `pendingRedeemQueue` to `redeemQueue`.
22 7. At T+4, we receive the USDC from BNY.
23 8. Fund operator calls `processRedeemQueue` (OPERATOR_ROLE) to burn Hybond tokens and
   transfer USDC to users (FIFO).
```

Express currently shares a single **priceOracle**.



<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/Express.sol#L605-L639>

```
605  /**
606  * @notice Process queued deposit requests
607  * @param _len Number of requests to process (0 = process all)
608  */
609  function processDepositQueue(uint256 _len) external onlyRole(MAINTAINER_ROLE) {
610  _len = _validateQueueProcessing(depositQueue.length(), _len);
611
612  for (uint256 count = 0; count < _len; ) {
613  bytes memory data = depositQueue.front();
614  (
615  address asset,
616  address sender,
617  address receiver,
618  uint256 netAssets,
619  uint256 feeAmt,
620  bytes32 prevId
621  ) = _decodeDepositData(data);
622
623  _validateKyc(sender, receiver);
624
625  depositQueue.popFront();
626  depositInfo[receiver][asset] -= netAssets;
627  unchecked {
628  ++count;
629  }
630
631  uint256 amount = convertFromUnderlying(asset, netAssets);
632  uint256 price = getPrice();
633  uint256 mintedAmount = _trim(Math.mulDiv(amount, 1e18, price));
634
635  token.mint(receiver, mintedAmount);
636
637  emit ProcessDeposit(asset, sender, receiver, netAssets, mintedAmount,
638  feeAmt, prevId);
639  }
```

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/-/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/Express.sol#L779-L800>

```

779  /**
780  * @notice Process pending redeems that have reached convertRedeemRequestsDelay
781  * @dev Applies current price to eligible pending requests and moves to final queue
782  * @param _len Number of pending requests to process (0 = all eligible)
783  */
784  function processPendingRedeems(uint256 _len) external onlyRole(OPERATOR_ROLE) {
785      uint256 maxToProcess = _validateQueueProcessing(pendingRedeemQueue.length(),
786      _len);
787      uint256 processed;
788      uint256 currentPrice = getPrice();
789      uint256 currentRatio = _sharesPerToken();
790      while (processed < maxToProcess && !pendingRedeemQueue.empty()) {
791          if (!_processSinglePendingRedeem(currentPrice, currentRatio)) {
792              break;
793          }
794          unchecked {
795              ++processed;
796          }
797      }
798      if (processed == 0) revert NoPendingRedeemsReady();
799  }
800  }

```

```

564  /**
565  * @notice Get fresh price from price feed with staleness check
566  * @return price The fresh price (normalized to 18 decimals)
567  */
568  function getPrice() public view returns (uint256 price) {
569      // If price oracle not set, return 1:1 ratio (1e18)
570      if (address(priceOracle) == address(0)) {
571          return 1e18;
572      }
573
574      (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound)
575      = priceOracle.latestRoundData();
576
577      // Validate price is positive
578      if (answer <= 0) revert InvalidPrice(answer);
579
580      // Check for stale price data
581      if (maxStalePeriod > 0 && block.timestamp - updatedAt > maxStalePeriod) {
582          revert StalePriceData(updatedAt, block.timestamp, maxStalePeriod);
583      }
584      return uint256(answer);
585  }

```



```
582 }
583
584 // Check for incomplete round data
585 if (answeredInRound < roundId) {
586     revert IncompleteRound(answeredInRound, roundId);
587 }
588
589 // Normalize price to 18 decimals
590 uint8 decimals = priceOracle.decimals();
591 price = uint256(answer);
592
593 if (decimals < 18) {
594     price = price * 10 ** (18 - decimals);
595 } else if (decimals > 18) {
596     price = price / 10 ** (decimals - 18);
597 }
598 // If decimals == 18, price remains unchanged
599 }
```

Status

① Acknowledged



[WP-I33] The Chainlink-compatible answer type is int256, which is inconsistent with the current PriceOracle implementation.

Informational

Issue Description

Chainlink interface definitions:

- `latestRoundData()` returns `(uint80 roundId, int256 answer, uint256 startedAt, uint256 updatedAt, uint80 answeredInRound)`: <https://docs.chain.link/data-feeds/api-reference/latestrounddata>
- `latestAnswer()` returns `(int256)`: <https://etherscan.io/address/0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419#readContract#F8>
- `AggregatorV2V3Interface`: <https://github.com/smartcontractkit/libocr/blob/a03701e2c02e2331921bfa6887e2257dea4e6084/contract/AggregatorV2V3Interface.sol>

The current `PriceOracle` implementation returns `uint256`.

When `PriceOracle.latestRoundData()` or `PriceOracle.latestAnswer()` returns a value greater than `uint256(type(int256).max)`, integrations relying on the standard `AggregatorV3Interface` will unexpectedly interpret the result as a negative number.

Furthermore, the return types for `PriceOracle.version()` and `PriceOracle.latestRound()` are also inconsistent with the Chainlink `AggregatorV2V3Interface`.

Additionally, `latestRound()` and `latestAnswer()` are marked as deprecated in the [Chainlink ETH / USD price feed](#). You may want to mark these functions as deprecated in `PriceOracle` or remove them entirely.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/67d457005a96f66960422e44009383bd2fa2b834/contracts/extension/PriceOracle.sol>

```
1  @@ 1,6 @@
7
8  /**
9  * @title PriceOracle
```



```
10  * @notice Upgradeable oracle contract for managing staged price updates with deviation
    controls
11  * @dev Uses UUPS upgradeable pattern for future improvements
12  *
13  * Key Features:
14  * - Two configurable price deviation limits for safety
15  * - Two-step price updates (proposal + confirmation)
16  * - Round-based price updates (Chainlink-compatible interface)
17  * - Role-based access control
18  * - Upgradeable via UUPS pattern
19  */
20  contract PriceOracle is Initializable, AccessControlEnumerableUpgradeable,
    UUPSUpgradeable {
21  @@ 21,115 @@
116
117  /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
118  VIEW FUNCTIONS
119  //////////////////////////////////////////////////////////////////////*/
120
121  function decimals() public view returns (uint8) {
122  return decimalsValue;
123  }
124
125  function latestRound() public view returns (uint80) {
126  return latestRoundValue;
127  }
128
129  function latestRoundData() public view returns (uint80, uint256, uint256, uint256,
    uint80) {
130  RoundData storage round = roundData[latestRoundValue];
131  return (round.roundId, round.answer, round.startedAt, round.updatedAt,
    round.answeredInRound);
132  }
133
134  function latestAnswer() public view returns (uint256) {
135  return roundData[latestRoundValue].answer;
136  }
137
138  @@ 138,285 @@
286  }
```



```
565  @@ 565,568 @@
569  function getPrice() public view returns (uint256 price) {
570  @@ 570,573 @@
574
575  (uint80 roundId, int256 answer, , uint256 updatedAt, uint80 answeredInRound)
    = priceOracle.latestRoundData();
576
577  @@ 577,599 @@
600  }
```

Status

✓ Fixed



[WP-N34] Initializable and UUPSUpgradeable in @openzeppelin/contracts-upgradeable have been deprecated since version 5.5.0.

Issue Description

Consider migrating to `Initializable` and `UUPSUpgradeable` from `@openzeppelin/contracts` instead.

<https://github.com/OpenZeppelin/openzeppelin-contracts-upgradeable/blob/v5.6.1/-CHANGELOG.md#deprecation>

Deprecation

- `Initializable` and `UUPSUpgradeable` are no longer transpiled. Aliases are present in the `@openzeppelin/contracts-upgradeable` package that redirect to the corresponding files in `@openzeppelin/contracts`. These aliases will be removed in the next major release. Developers are advised to update their imports to get these files directly from the `@openzeppelin/contracts` package. [#5941](#)

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/PriceOracle.sol>

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.20;
3
4  import
    "@openzeppelin/contracts-upgradeable/access/extensions/AccessControlEnumerableUpgradeable.sol"-
    ;
5  import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
6  import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
7
8  @@ 8,19 @@
20  contract PriceOracle is Initializable, AccessControlEnumerableUpgradeable,
    UUPSUpgradeable {
21  @@ 21,304 @@
305  }
```



<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/ab3c54c81f0f15e3a22baee15bbcf18158016245/contracts/extension/Express.sol>

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
5 import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
6 import
  "@openzeppelin/contracts-upgradeable/access/extensions/AccessControlEnumerableUpgradeable.sol"-
  ;
7 import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
8 import "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
9 import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
10 import { Math } from "@openzeppelin/contracts/utils/math/Math.sol";
11
12 import "../ExpressPausable.sol";
13 import "../DepositRedeemLimiter.sol";
14 import "../DoubleQueueModified.sol";
15 import { IToken } from "../interfaces/IToken.sol";
16 import "../interfaces/IAssetRegistry.sol";
17 import "../interfaces/IPriceFeed.sol";
18
19 @@ 19,36 @@
37
38 contract Express is
39     Initializable,
40     UUPSUpgradeable,
41     AccessControlEnumerableUpgradeable,
42     ExpressPausable,
43     DepositRedeemLimiter
44 {
```

Status

✓ Fixed



[WP-G35] Values that do not change, like PriceOracle.decimalsValue, should be declared as immutable to avoid unnecessary SLOAD gas costs.

Gas

Issue Description

The `decimalsValue` is only modified within the `initializer` function and nowhere else.

Consider using an `immutable` variable and setting the value in the implementation contract's `constructor`. This applies to both the initial deployment and any subsequent upgrades to a new implementation contract.

<https://github.com/OpenEdenHQ/openeden.hybond.audit/blob/67d457005a96f66960422e44009383bd2fa2b834/contracts/extension/PriceOracle.sol#L8-L286>

```
8  /**
9   * @title PriceOracle
10  * @notice Upgradeable oracle contract for managing staged price updates with deviation
    controls
11  * @dev Uses UUPS upgradeable pattern for future improvements
12  *
13  * Key Features:
14  @@ 14,17 @@
18  * - Upgradeable via UUPS pattern
19  */
20  contract PriceOracle is Initializable, AccessControlEnumerableUpgradeable,
    UUPSUpgradeable {
21  @@ 21,38 @@
39
40  uint8 private decimalsValue;
41
42  @@ 42,69 @@
70
71  /// @custom:oz-upgrades-unsafe-allow constructor
72  constructor() {
73  _disableInitializers();
74  }
75
76  @@ 76,84 @@
85  function initialize(
86  uint8 _decimals,
```



```
87  uint256 _relativeMaxDeviation,  
88  uint256 _absoluteMaxDeviation,  
89  uint256 _initPrice,  
90  uint256 _referencePrice,  
91  address _admin  
92  ) external initializer {  
93  @@ 93,99 @@  
100  
101  decimalsValue = _decimals;  
102  @@ 102,114 @@  
115  }  
116  
117  /*/////////////////////////////////////////////////////////////////  
118  VIEW FUNCTIONS  
119  //////////////////////////////////////////////////////////////////////*/  
120  
121  function decimals() public view returns (uint8) {  
122  return decimalsValue;  
123  }  
124  
125  @@ 125,285 @@  
286  }
```

Status

✓ Fixed



Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.